claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3\agent-a1e035bd021a14eaf.jsonl`
- SHA-256: `8198fbe5532d2561d86e91129fcbe40d20f9384ca2e4029fe5c73fdb9e4d3b95`
- Source modified: `2026-06-18T01:31:55+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b7f012f8-5c3`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:27:48.620Z)

```
You are an adversarial bug-hunter in a mature Python repo (badminton-highlight-indexer:
FastAPI + SQLite + ffmpeg + GPU CV/AI rally detector). Read code READ-ONLY in this worktree
(pinned to origin/master ? DO NOT modify anything):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base

Hunt ONLY for REAL, REACHABLE defects a maintainer would agree is a bug and want fixed:
- crash on a reachable input (unguarded dict/index/attr access, None deref,
KeyError/IndexError),
- wrong logic / off-by-one / inverted condition / wrong operator,
- incorrect error handling (catching too much/little, swallowing, wrong exception type),
- resource leak (unclosed file/db/subprocess), TOCTOU, mutable-default-arg,
- contract mismatch between a function and its callers/tests,
- wrong default / unit / sign that produces incorrect output.

HARD RULES:
- PRECISION over recall. Report a finding ONLY if you traced the actual code and a concrete
  trigger reaches it. No style, no "could be cleaner", no speculative "might", no perf-nits,
  no missing-test observations, no naming. If you are not sure it is a real bug, DROP it.
- For each finding give an EXACT file path + line, the bug_class, a CONCRETE trigger (the input
  or call path that hits it), why it is wrong, severity, and a minimal proposed fix.
- This repo has 848 passing tests and several hardening sweeps ? easy bugs are already gone.
  Expect FEW or ZERO real bugs in your area. Returning an empty candidates list is a VALID,
  respected answer. Do NOT manufacture findings to look productive.
- EXCLUDE these files entirely (just fixed / fix already in flight): backend/eval/ablation.py,
  backend/eval/fusion_compare.py, backend/eval/fusion_audio.py.
- Verify your trigger by reading the surrounding code and any related test, not from memory.

YOUR AREA: scripts/ (EXCLUDING
scripts/nightly_regression.py ? owned by a separate PR), training/, and any FastAPI app/router/
endpoint + request validation in backend/ (search for FastAPI/APIRouter). Focus on argparse
defaults, CLI exit codes, request validation, and path/file handling.
```

2. assistant (2026-06-18T01:27:51.492Z)

```
I'll hunt for bugs in my assigned area: scripts/ (excluding nightly_regression.py), training/,
and FastAPI app/router/endpoint code in backend/. Let me start by exploring the structure.
```

3. user (2026-06-18T01:27:53.740Z)

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
---SCRIPTS---
README.md
doctor.py
nightly_regression.ps1
```

```
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
run_process_and_stitch.py
---TRAINING---
README.md
__init__.py
gen0
requirements-train.txt
```

4. user (2026-06-18T01:27:54.805Z)

```
Found 1 file
.claude\worktrees\hunt-base\backend\main.py
```

5. assistant (2026-06-18T01:27:56.596Z)

Let me look at the FastAPI app and the scripts in parallel.

6. user (2026-06-18T01:27:58.182Z)

```
1       import sys
2       import os
3       import subprocess
4       import shutil
5
6       # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7       # root (requirements.txt, config.json). Resolve the root from this file's
8       # location so it works regardless of the caller's cwd.
9       REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11      def check_command(cmd):
12          """Check if a system command is available."""
13          return shutil.which(cmd) is not None
14
15      def run_pip_install():
16          """Install required Python packages."""
17          print("Installing Python packages from requirements.txt...")
18
19          # Check if NVIDIA GPU is present
20          has_gpu = check_command("nvidia-smi")
21
22          requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23          pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25          if has_gpu:
26              print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
27              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
28          else:
29              print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
30              pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
31
32          try:
33              subprocess.check_call(pip_cmd)
34              print("[SUCCESS] Python packages installed.")
35              return True
36          except subprocess.CalledProcessError as e:
37              print(f"[ERROR] Failed to install Python packages: {e}")
38              return False
39
40      def init_default_config():
41          """Initialize a default config.json if it doesn't exist.
```

```
42
43          Now delegates to the canonical backend.config loader so that defaults
44          stay in sync with the Pydantic models (single source of truth).
45          """
46          try:
47              from backend.config import save_default_config
48          except Exception as e:
49              print(f"[ERROR] Could not import backend.config loader: {e}")
50              print("[INFO] Falling back to no-op for config initialization.")
51              return
52
53          config_path = os.path.join(REPO_ROOT, "config.json")
54          if os.path.exists(config_path):
55              print(f"[INFO] {config_path} already exists. Skipping initialization.")
56              return
57
58          try:
59              saved_path = save_default_config(config_path)
60              print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
61          except Exception as e:
62              print(f"[ERROR] Failed to create config.json via loader: {e}")
63
64      def run_diagnostics():
65          """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
66          print("=" * 50)
67          print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
68          print("=" * 50)
69
70          # 1. Check Python Version
71          print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
72          if sys.version_info.major < 3 or (sys.version_info.major == 3 and
sys.version_info.minor < 8):
73              print("[WARNING] Python 3.8+ is highly recommended!")
74          else:
75              print("[PASS] Python version check passed.")
76
77          print("-" * 50)
78
79          # 2. Check FFmpeg / FFprobe
80          ffmpeg_ok = check_command("ffmpeg")
81          ffprobe_ok = check_command("ffprobe")
82
83          if ffmpeg_ok:
84              print("[PASS] FFmpeg is installed and available in PATH.")
85          else:
86              print("[FAIL] FFmpeg is NOT found in your PATH.")
87
88          if ffprobe_ok:
89              print("[PASS] FFprobe is installed and available in PATH.")
90          else:
91              print("[FAIL] FFprobe is NOT found in your PATH.")
92
93          if not (ffmpeg_ok and ffprobe_ok):
94              print("\n" + "!" * 50)
95              print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
96              print("How to install FFmpeg on Windows:")
97              print("  Option 1: Open PowerShell as Administrator and run:")
98              print("            winget install Gyan.FFmpeg")
99              print("  Option 2: Download the build from https://ffmpeg.org/download.html")
100             print("            and add its 'bin' directory to your System PATH environment
variable.")
101             print("!" * 50 + "\n")
102         else:
```

```
            print("[SUCCESS] All essential media command line tools are ready.")

        print("-" * 50)

        # 3. Check ML / GPU (Optional but good)
        try:
            import torch
            cuda_ok = torch.cuda.is_available()
            if cuda_ok:
                print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
            else:
                print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
        except ImportError:
            print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")

        print("=" * 50)
        return ffmpeg_ok and ffprobe_ok

    if __name__ == "__main__":
        init_default_config()
        media_ok = run_diagnostics()
        run_pip_install()
        print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")

```

7. user (2026-06-18T01:27:58.401Z)

```
import os
import sys
import argparse
import logging
import threading
import traceback
import uuid
from concurrent.futures import ThreadPoolExecutor
import uvicorn
from dotenv import load_dotenv

load_dotenv()

# Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
    datefmt='%H:%M:%S'
)
logger = logging.getLogger(__name__)
from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Dict, Any, Optional

from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from backend.pipeline.validators.base import registry
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
```

```
    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  # new typed config
    from backend.results import Failure  # standardized error/result contract
    from backend.reporting import emit_indexer_report

    app = FastAPI(title="Sports Highlight Indexer API")

    # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True is
    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=False,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly (now under api/ per approved structure)
    os.makedirs("backend/api/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/api/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in backend/api/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    _job_executor: Optional[ThreadPoolExecutor] = None

    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="jobworker")
        return _job_executor

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
```

```
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        video_id = compute_video_id(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
```

```
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response
```

```
212
213             segments = result.value if hasattr(result, "value") else result
214             notify("finalizing")
215             _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217             response = {
218                 "video_id": video_id,
219                 "status": "success",
220                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
221                 "results": [r.model_dump() for r in val_results],
222                 "play_segments": segments,
223             }
224             return response
225         finally:
226             # Always emit the per-video observability report (next to the video, or to
227             # report.output_dir). Never raises. Surface the paths in the response.
228             paths = emit_indexer_report(
229                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
230                 val_results=val_results, val_context=val_context,
231                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
232                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
233             )
234             if paths and isinstance(response, dict):
235                 response["reports"] = paths
236
237
238     class JobWaiting(Exception):
239         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
240         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
241         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
242         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
243         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
244
245         The wiring is additive: the production segmenter path never raises this today (zero
246         behaviour change), so a job runs exactly as before. The hook is what a later slice
247         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
248         """
249
250         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
251             super().__init__(f"job parked for {delay_sec:.0f}s")
252             self.delay_sec = max(0.0, float(delay_sec))
253             self.progress = progress
254
255
256     def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
257         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
258         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
259         the original submit. The row stores exactly the ProcessRequest fields."""
260         return ProcessRequest(
261             video_path=job["video_path"],
262             player_pool=job.get("player_pool"),
263             max_frames=job.get("max_frames"),
264             segmenter=job.get("segmenter"),
265         )
266
267
268     def _run_claimed_job(job: Dict[str, Any]) -> None:
269         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
270         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
```

```
271
272         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
273         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
274         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
275         job self-scheduled and will be re-claimed when `next_poll_at` is due.
276         """
277         job_id = job["id"]
278         req = _job_to_request(job)
279         try:
280             result = _execute_processing(
281                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
282             if result.get("video_id"):
283                 db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285         except JobWaiting as w:
286             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289         except Exception as e:
290             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294     def _drain_due_jobs() -> int:
295         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
296         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
298         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
299         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
300         a single-worker box keeps making progress with no central timer (the DB is the
clock).
301         Returns the number of jobs that reached a terminal/parked state this tick.
302         """
303         ran = 0
304         while True:
305             job = db_instance.claim_due_job()
306             if job is None:
307                 break
308             ran += 1
309             _run_claimed_job(job)
310         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
311         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
312         # on the next submit/drain. Best-effort ? never raises into the worker.
313         _schedule_next_drain_if_waiting()
314         return ran
315
316
317     def _schedule_next_drain_if_waiting() -> None:
318         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319         self-scheduled job resumes with no further client submit. The drain itself re-checks
320         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
321         try:
322             delay = db_instance.seconds_until_next_due()
323         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
324             logger.error(f"Could not compute next-due delay: {e}")
325             return
```

```
        if delay is None:
            return  # nothing waiting
        # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
        _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))


    def _arm_wake_timer(delay_sec: float) -> None:
        """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
        scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
        monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
        timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
        timer.daemon = True
        timer.start()


    #: A parked job further out than this is re-checked by a capped wake rather than one
long
    #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
    _MAX_DRAIN_WAKE_SEC = 60.0


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
```

```
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
    # proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
    # under security.upload_dir, then submitted to /api/process by the returned path.
    # Upload state is in-process by design (this is the single-box alpha): a server restart
    # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    # arrives with PR-3 identity scoping.

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        sec = getattr(global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @app.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
```

```
448             raise HTTPException(status_code=400, detail=str(e)) from e
449         ext = os.path.splitext(name)[1].lower().lstrip(".")
450         if ext not in exts:
451             raise HTTPException(status_code=400,
452                                 detail=f"Extension '.{ext}' not allowed. Allowed:
{sorted(exts)}")
453         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
454             raise HTTPException(status_code=413,
455                                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
456         os.makedirs(upload_dir, exist_ok=True)
457         upload_id = uuid.uuid4().hex
458         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
459         open(tmp_path, "wb").close()
460         with _uploads_lock:
461             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
462         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
463                 "next_part": f"/api/upload/{upload_id}/part/0"}
464
465
466     @app.put("/api/upload/{upload_id}/part/{index}")
467     async def upload_part(upload_id: str, index: int, request: Request):
468         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
469         _, max_bytes, part_bytes, _ = _upload_cfg()
470         with _uploads_lock:
471             st = _uploads.get(upload_id)
472         if st is None:
473             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
474         if index != st["parts"]:
475             raise HTTPException(status_code=409,
476                                 detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
477
478         received = 0
479         overflow = None
480         with open(st["tmp_path"], "ab") as fh:
481             async for chunk in request.stream():
482                 received += len(chunk)
483                 if received > part_bytes:
484                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                     break
486                 if st["bytes"] + received > max_bytes:
487                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                     break
489                 fh.write(chunk)
490         if overflow:
491             _abort_upload(upload_id)
492             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493
494         with _uploads_lock:
495             st["parts"] += 1
496             st["bytes"] += received
497         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498
499
500     @app.post("/api/upload/{upload_id}/complete")
501     def upload_complete(upload_id: str, req: UploadCompleteRequest):
502         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503         upload_dir, _, _, _ = _upload_cfg()
```

```
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(status_code=409,
                                detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
        final_path = os.path.join(upload_dir, st["filename"])
        if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
            stem, ext = os.path.splitext(st["filename"])
            final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
        os.replace(st["tmp_path"], final_path)
        with _uploads_lock:
            _uploads.pop(upload_id, None)
        video_id = compute_video_id(final_path)
        return {"path": os.path.abspath(final_path), "video_id": video_id,
                "size_bytes": os.path.getsize(final_path),
                "next": "POST /api/process with this path"}


    @app.delete("/api/upload/{upload_id}")
    def upload_abort(upload_id: str):
        """Abort an in-flight upload and delete its partial file."""
        with _uploads_lock:
            known = upload_id in _uploads
        if not known:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        _abort_upload(upload_id)
        return {"status": "aborted", "upload_id": upload_id}


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
```

```
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "

f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
```

```
watermark=stitching.watermark)
622         try:
623             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
624             return {"status": "success", "filepath": out_path}
625         except Exception as e:
626             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
627
628     # --- CLI Debug Utility & Orchestration ---
629     def run_cli_diagnose_clip(video_path: str, timestamp: float):
630         """CLI debugging utility to examine segment properties around a timestamp."""
631         print("=" * 60)
632         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
633         print("=" * 60)
634
635         cfg = load_config()  # legacy wrapper still works, returns dict for now
636
637         # 1. Validation check diagnostics
638         print("[DIAGNOSTIC] Running validation framework...")
639         results = registry.run_all(video_path, cfg)
640         for r in results:
641             status_symbol = "[PASS]" if r.passed else "[FAIL]"
642             print(f"  {status_symbol} {r.validator_name}: {r.message}")
643             if r.details:
644                 print(f"         Details: {r.details}")
645
646         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
647         print("-" * 60)
648         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
649         import cv2
650         cap = cv2.VideoCapture(video_path)
651         if not cap.isOpened():
652             print("[FAIL] OpenCV could not open video.")
653             return
654
655         fps = cap.get(cv2.CAP_PROP_FPS)
656         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
657
658         start_frame = max(0, int((timestamp - 3.0) * fps))
659         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
660
661         # Measure frame-by-frame changes in sample region
662         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
663         prev_gray = None
664         motions = []
665
666         for _ in range(start_frame, end_frame):
667             ret, frame = cap.read()
668             if not ret:
669                 break
670             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671             gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673             if prev_gray is not None:
674                 diff = cv2.absdiff(prev_gray, gray)
675                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677                 motions.append(motion_ratio)
678             prev_gray = gray
679
680         cap.release()
681
682         if motions:
683             avg_motion = sum(motions) / len(motions)
```

```
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess
            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts", "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
```

```
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
        # the executor is in-process. Mark them failed so pollers see a terminal state.
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            if not os.path.exists(args.video_path):
```

```
794                 print(f"[FAIL] Video file not found: {args.video_path}")
795                 return
796
797             video_id = compute_video_id(args.video_path)
798             was_reingest = db_instance.get_video(video_id) is not None
799
800             # Validate (with-context variant surfaces ffprobe_meta for the report)
801             print("[CLI] Validating...")
802             val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803             failures = [r for r in val_results if not r.passed]
804
805             # Save video status
806             status = "failed" if failures else "processed"
807             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810             segmenter_actual = None
811             segmentation_ran = False
812             try:
813                 print("\n" + "="*40)
814                 print("VALIDATION SUMMARY")
815                 print("="*40)
816                 for r in val_results:
817                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
818                     print(f"{status_symbol} {r.validator_name}: {r.message}")
819                 print("="*40 + "\n")
820
821                 if failures:
822                     print("[FAIL] Video failed checks. Indexing halted.")
823                     return
824
825                 # Index
826                 segmenter_cls = segmenter_registry.get(seg_name)
827                 if not segmenter_cls:
828                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
829                     return
830
831                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832                 segmenter_actual = seg_name
833                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834                 segmenter = segmenter_cls(db_instance, global_config)
835                 result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836                 segmentation_ran = True
837
838                 if isinstance(result, Failure):
839                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
840                     print("[FAIL] Indexing halted for video.")
841                     segments = []
842                 else:
843                     segments = result.value if hasattr(result, "value") else result
844                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
845                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
847             finally:
848                 # Always emit the per-video observability report. Never raises.
849                 paths = emit_indexer_report(
850                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851                     val_results=val_results, val_context=val_context,
```

```
852                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854                 )
855                 if paths:
856                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
859             return
860
861         # Start Local Server Mode
862         if args.server or not args.video_path:
863             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
864             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
865             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
866             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
867             _get_executor().submit(_drain_due_jobs)
868             uvicorn.run(app, host="127.0.0.1", port=args.port)
869
870     if __name__ == "__main__":
871         main()
872
```

8. assistant (2026-06-18T01:28:00.549Z)

Let me look at the remaining scripts and training code.

9. user (2026-06-18T01:28:01.974Z)

```
1       """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3       Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4       processes each video through a segmenter (so detections land in the indexer DB),
5       scores each against its ground-truth rally CSV, and prints per-video plus
6       corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8       Examples
9       --------
10      Free CPU baseline on a single video (validate the loop):
11          python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12              --segmenter motion --only shuttleset_1
13
14      Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15          python run_phase3_eval.py --manifest .../manifest.json --segmenter yolo_hybrid \
16              --json output/phase3_report.json
17
18      Re-score only (videos already processed), no re-segmentation:
19          python run_phase3_eval.py --manifest .../manifest.json --score-only
20      """
21
22      import os
23      import sys
24      import json
25      import time
26      import argparse
27      import logging
28
29      from dotenv import load_dotenv
30      load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32      from backend.infrastructure.database import Database
33      from backend.config import load_config
```

```
    from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
    # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
    from backend.pipeline.segmenters import segmenter_registry
    from backend.eval.annotations import load_annotations
    from backend.eval.harness import evaluate, format_report_text, report_to_dict
    from backend.eval import batch

    logger = logging.getLogger(__name__)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            description="Batch process + evaluate a collector export manifest (Phase 3).",
            formatter_class=argparse.RawDescriptionHelpFormatter,
        )
        p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
        p.add_argument("--videos-root", default=None,
                       help="Root for resolving manifest relative local_paths "
                            "(default: the collector repo root, two levels above the
manifest).")
        p.add_argument("--segmenter", default="motion",
                       help="Segmenter to run (default: motion ? free/CPU). "
                            "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
        p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
                       help="Which prediction stage(s) to score (default: auto by
segmenter).")
        p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
        p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
        p.add_argument("--only", action="append", default=None,
                       help="Only this video_id (repeatable).")
        p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
        p.add_argument("--max-frames", type=int, default=None,
                       help="Cap segmenter frames (fast validation).")
        p.add_argument("--score-only", action="store_true",
                       help="Skip segmentation; only score detections already in the DB.")
        p.add_argument("--reprocess", action="store_true",
                       help="Re-run segmentation even if the video already has segments.")
        p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
        p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
        p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
        return p


    def main(argv=None) -> int:
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        args = build_parser().parse_args(argv)

        manifest_path = os.path.abspath(args.manifest)
        if not os.path.exists(manifest_path):
            print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
            return 2
        manifest_dir = os.path.dirname(manifest_path)
        videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))

        with open(manifest_path) as f:
            manifest = json.load(f)
        entries = manifest.get("eval_sets", [])

```

```
90          cfg = load_config()  # typed AppConfig (same object the live pipeline feeds
segmenters)
91          db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92          os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93          db = Database(db_path)
94
95          stages = batch.default_stages_for(args.segmenter, args.stage)
96
97          # Resolve the segmenter class up front (unless purely scoring).
98          segmenter = None
99          if not args.score_only:
100             seg_cls = segmenter_registry.get(args.segmenter)
101             if not seg_cls:
102                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
103                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
104                 return 2
105             segmenter = seg_cls(db, cfg)
106
107         # Filter entries.
108         targets = []
109         for e in entries:
110             if args.only and e["video_id"] not in args.only:
111                 continue
112             targets.append(e)
113         if args.limit:
114             targets = targets[: args.limit]
115
116         per_video = []
117         skipped = []
118         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
119
120         for e in targets:
121             vid = e["video_id"]
122             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
123             csv_path = os.path.join(manifest_dir, e["csv"])
124
125             if not video_path or not os.path.exists(video_path):
126                 skipped.append((vid, "no video file"))
127                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
128                 continue
129             if not os.path.exists(csv_path):
130                 skipped.append((vid, "no GT csv"))
131                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
132                 continue
133
134             video_id = get_file_md5(video_path)
135
136             # Segment unless score-only or already present (and not --reprocess).
137             if not args.score_only:
138                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
139                 if already and not args.reprocess:
140                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
141                 else:
142                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
143                     db.add_video(video_id, video_path, "processed", [])
144                     t0 = time.time()
145                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
146                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
147
148             gt = load_annotations(csv_path)
```

```
149             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
150             print(format_report_text(report, show_failures=args.show_failures))
151             print()
152             d = report_to_dict(report)
153             d["collector_video_id"] = vid
154             per_video.append(d)
155
156         agg = batch.aggregate(per_video, stages)
157         print(batch.format_aggregate(agg))
158         if skipped:
159             print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
160
161         if args.json_out:
162             with open(args.json_out, "w") as f:
163                 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
164                            "aggregate": agg, "per_video": per_video,
165                            "skipped": skipped}, f, indent=2)
166             print(f"\nJSON report written to {args.json_out}")
167
168         return 0
169
170
171     if __name__ == "__main__":
172         raise SystemExit(main())
173
```

10. user (2026-06-18T01:28:02.262Z)

```
1       import os
2       import json
3       import logging
4       from dotenv import load_dotenv
5
6       load_dotenv()
7
8       # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9       # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10      # (logging.verbose_http) once the config is loaded in main().
11      from backend.utils.logging_setup import setup_logging
12      setup_logging()
13
14      from backend.infrastructure.database import Database
15      from backend.config import load_config
16      from backend.utils.hashing import compute_video_id
17      from backend.pipeline.segmenters import segmenter_registry
18      from backend.pipeline.stitcher.compiler import VideoCompiler
19      from backend.tools.export import format_rally_summary
20
21      logger = logging.getLogger(__name__)
22
23      def main():
24          video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
25          output_dir = r"C:\Users\avidu\Videos\Testing"
26          output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
27
28          print("Loading config...")
29          cfg = load_config()  # typed AppConfig ? single source of defaults
30          # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
31          setup_logging(verbose_http=cfg.logging.verbose_http)
32
33          print("Verifying files exist...")
34          if not os.path.exists(video_path):
```

```
35              print(f"Error: Video file not found at {video_path}")
36              return
37
38          os.makedirs(cfg.output.output_dir, exist_ok=True)
39          db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
40          db = Database(db_path)
41
42          # Calculate MD5 of the video to uniquely identify it
43          import time
44          print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
45          t0 = time.time()
46          video_id = compute_video_id(video_path)
47          print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
48
49          segments = []
50          cached_video = db.get_video(video_id)
51          cache_hit = False
52
53          if cached_video and cached_video.get("status") == "processed":
54              segments = db.query_play_segments(video_id, {})
55              if len(segments) > 0:
56                  print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
57                  choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
58                  if choice == '1':
59                      print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
60                      cache_hit = True
61                  else:
62                      print("[CACHE] Ignoring cache...", flush=True)
63              else:
64                  print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
65
66          if not cache_hit:
67              print("[CACHE] Initializing clean indexing...", flush=True)
68              db.clear_video_data(video_id)
69
70              print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
71              db.add_video(video_id, video_path, "processed", [])
72
73              print("Running segmenter & indexing...", flush=True)
74              seg_name = cfg.indexing.default_segmenter
75              segmenter_cls = segmenter_registry.get(seg_name)
76              if not segmenter_cls:
77                  print(f"[FAIL] Segmenter '{seg_name}' not found.")
78                  return
79              print(f"Using segmenter: {seg_name}", flush=True)
80              segmenter = segmenter_cls(db, cfg)
81              segments = segmenter.process_video(video_id, video_path)
82              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
83
84          if not segments:
85              print("No play segments detected. Cannot compile highlights.")
86              return
87
88
89          end_padding = cfg.stitching.end_padding
90          begin_padding = cfg.stitching.begin_padding
91          min_shot_count = cfg.stitching.min_shot_count
92
93          if cache_hit:
```

```
            print("\n--- Stitching Customizations ---")
            try:
                bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
                if bpad_input:
                    begin_padding = float(bpad_input)

                pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
                if pad_input:
                    end_padding = float(pad_input)

                shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
                if shot_input:
                    min_shot_count = int(shot_input)
            except ValueError:
                print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")

        print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)

        # Filter segments where shot_count >= min_shot_count
        time_pairs = []
        skipped_count = 0
        for s in segments:
            meta = s.get("metadata", {})
            if isinstance(meta, str):
                try:
                    meta = json.loads(meta)
                except Exception:
                    meta = {}
            shot_count = meta.get("shot_count", 0)
            if shot_count >= min_shot_count:
                time_pairs.append((s["start_time"], s["end_time"]))
            else:
                skipped_count += 1

        if skipped_count > 0:
            print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
        print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)

        # End-of-run rally table for manual QA ? sourced from the produced play_segments so
        # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
        # aligned rows) so it survives even if the table is the last thing the owner scans.
        def _log_rally_summary() -> None:
            logger.info("\n%s", format_rally_summary(segments))

        if not time_pairs:
            print("No play segments passed the filtering criteria. Cannot compile
highlights.")
            _log_rally_summary()
            return

        compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding,
                                 watermark=cfg.stitching.watermark)
        try:
            out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
            print(f"Stitched highlight video saved to: {out_path}", flush=True)
        except Exception as e:
            print(f"Error during stitching: {e}", flush=True)
```

```
150
151         _log_rally_summary()
152
153     if __name__ == "__main__":
154         main()
155
```

11. assistant (2026-06-18T01:28:05.089Z)

Now let me look at the training/ directory and the batch helper used by phase3.

12. user (2026-06-18T01:28:06.768Z)

```
./gen0/harness.py
./gen0/__init__.py
./__init__.py
```

13. user (2026-06-18T01:28:06.849Z)

```
1       """Batch evaluation across a collector export manifest.
2
3       Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4       sports-data-collector's `curate export`, (optionally) process each video through
5       a segmenter so detections land in the indexer DB, then score every video's
6       detected rallies against its ground-truth CSV and aggregate the results.
7
8       This module holds the *pure* pieces (path resolution, stage selection, metric
9       aggregation) so they can be unit-tested without touching video files, the GPU,
10      or any API. The orchestration/side-effects live in `scripts/run_phase3_eval.py`.
11      """
12
13      import os
14      from typing import Dict, List, Optional
15
16
17      # Segmenters that only ever populate the `final` play_segments table (no raw
18      # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19      _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22      def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23          """Resolve a manifest `local_path` to an absolute path.
24
25          The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
26          to its own repo root and with Windows separators. Normalise separators, strip
27          a leading ``./``, and resolve relative paths against `videos_root` (the
28          collector root). Returns None for a missing/empty path.
29          """
30          if not local_path:
31              return None
32          p = local_path.replace("\\", "/")
33          if p.startswith("./"):
34              p = p[2:]
35          # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36          # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37          # This keeps collector manifest paths working in tests and mixed environments.
38          if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39              return os.path.normpath(p)
40          return os.path.normpath(os.path.join(videos_root, p))
41
42
43      def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44          """Pick which prediction stages to score.
45
```

```
        ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
        but only ``final`` for segmenters that don't emit candidates. An explicit
        request ('candidates' | 'final' | 'both') overrides the heuristic.
        """
        if requested == "both":
            return ["candidates", "final"]
        if requested in ("candidates", "final"):
            return [requested]
        # auto
        if segmenter in _FINAL_ONLY_SEGMENTERS:
            return ["final"]
        return ["candidates", "final"]


    def _safe_div(num: float, den: float) -> float:
        return num / den if den else 0.0


    def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
        """Aggregate per-video stage scores into corpus-level metrics.

        `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
        ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
        compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
        a TP-weighted mean IoU, plus the macro mean of per-video F1.
        """
        out: Dict[str, dict] = {}
        for stage in stages:
            tp = fp = fn = 0
            iou_weighted = 0.0
            f1s: List[float] = []
            n_videos = 0
            for rep in per_video:
                sr = rep.get("stages", {}).get(stage)
                if not sr:
                    continue
                s = sr["score"]
                tp += s["true_positives"]
                fp += s["false_positives"]
                fn += s["false_negatives"]
                iou_weighted += s["mean_iou"] * s["true_positives"]
                f1s.append(s["f1"])
                n_videos += 1
            precision = _safe_div(tp, tp + fp)
            recall = _safe_div(tp, tp + fn)
            out[stage] = {
                "videos": n_videos,
                "tp": tp, "fp": fp, "fn": fn,
                "micro_precision": precision,
                "micro_recall": recall,
                "micro_f1": _safe_div(2 * precision * recall, precision + recall),
                "macro_f1": _safe_div(sum(f1s), len(f1s)),
                "mean_iou": _safe_div(iou_weighted, tp),
            }
        return out


    def format_aggregate(agg: Dict[str, dict]) -> str:
        """Human-readable corpus-level summary table."""
        lines = [
            "=" * 78,
            " CORPUS AGGREGATE",
            "=" * 78,
            f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
            f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
```

```
            "-" * 78,
        ]
        for stage, a in agg.items():
            lines.append(
                f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
                f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
                f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
            )
        lines.append("=" * 78)
        return "\n".join(lines)

```

14. assistant (2026-06-18T01:28:08.939Z)

Let me read the training harness, which is the main training code.

15. user (2026-06-18T01:28:09.987Z)

```
1       """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3           python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4               --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6       Productionizes the rally_seq prototype per ADR-008:
7       - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8         honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9         the FINAL model on all videos.
10      - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11            <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12            <out>/<version>/manifest.json   (the reviewable contract: feature-schema version,
13                                             calibration, training lineage + GT hashes,
14                                             eval results, attestations, gate verdict)
15      - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16        (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17        (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18        undefined => no self-declared victory). The product side owns final admission
19        (backend/eval/regression.py); this harness never grades itself into serving.
20
21      Training may import backend.* (one-way wall); backend must never import training.*.
22      """
23      from __future__ import annotations
24
25      import argparse
26      import hashlib
27      import json
28      import os
29      import subprocess
30      import time
31      from math import comb
32      from typing import Dict, List, Optional
33
34      import numpy as np
35
36      from backend.eval.classifier import LogisticRegression
37      from backend.eval.gt_loader import load_gt_intervals
38      from backend.eval.regression import gt_hash
39      from backend.eval.rally_seq_proto import (
40          labels_for, run as lovo_run, _seg_f1,
41      )
42      from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
43      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
44
```

```
    # Post-processing constants ? part of the model's identity, recorded in the manifest.
    CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
    THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]


    def _git_sha() -> str:
        try:
            return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
                                  text=True, timeout=10).stdout.strip() or "unknown"
        except Exception:
            return "unknown"


    def _sha256(path: str) -> str:
        h = hashlib.sha256()
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(65536), b""):
                h.update(chunk)
        return h.hexdigest()


    def sign_test(learned: List[float], floor: List[float]):
        """Paired one-sided sign test: is learned > floor more often than chance?

        At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
        wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
        pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv]  # ties drop,
standard
        wins = sum(1 for lv, fv in pairs if lv > fv)
        n = len(pairs)
        p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
        return wins, n, p


    def train_final(specs: List[Dict]):
        """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).

        The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
        vids = []
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
            X, times = features(arr, float(s["fps"]))
            gts = load_gt_intervals(s["labels"], strict=False)
            vids.append({"name": s["name"], "X": X, "times": times,
                         "y": labels_for(times, gts), "gts": gts})
        Xall = np.vstack([v["X"] for v in vids])
        yall = np.concatenate([v["y"] for v in vids])
        clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
        clf.fit(Xall, yall, feature_names=FEAT_NAMES)
        thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
        return clf, float(thr)


    def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
        """Pre-flight ship-gate.

        Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
        Floor (necessary): heuristic LOVO ~0.480 (from quality table).
        Multi-court ref (makes owned \"worth serving\" per data): 0.66.
        Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
        multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
        Beating floor + sign test necessary; C3 main commercial blocker.
```

```
        ship=True only when all non-C3 blocks clear (honesty first).
        """
        reasons: List[str] = []
        floor_passed = None
        sign = None
        if floor:
            floor_mean = float(floor["mean_f1"])
            floor_passed = eval_res["mean_f1"] > floor_mean
            if not floor_passed:
                reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
            per = floor.get("per_video", {})
            paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
            if paired:
                wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
                sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
                if p >= 0.05:
                    reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
                                   f"? grow the corpus before claiming superiority")
        else:
            reasons.append("no floor baseline provided ? nothing to beat")

        # Provisional blocker (per owner review on #170; see #172 for calibration).
        # (The number itself is not the blocker; calibration tracked in #172.)
        reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
        reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
        # ship remains False until C3 resolved + other gates (by design).
        return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}


    def run(manifest_path: str, out_dir: str, version: str,
            floor_path: Optional[str] = None) -> dict:
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        floor = None
        if floor_path:
            with open(floor_path, encoding="utf-8") as f:
                floor = json.load(f)

        # 1) Honest LOVO eval (the number that matters).
        eval_res = lovo_run(specs)

        # 2) Final model on all videos.
        clf, thr = train_final(specs)

        # 3) Gate (pre-flight).
        verdict = gate_verdict(eval_res, floor)

        # 4) Artifact bundle.
        bundle_dir = os.path.join(out_dir, version)
        os.makedirs(bundle_dir, exist_ok=True)
        weights_path = os.path.join(bundle_dir, "weights.json")
        payload = clf.to_dict()
        payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
        payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
        with open(weights_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)

        manifest = {
            "artifact": {
```

```
                "name": "rally_tal_gen0", "version": version,
                "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
                "git_sha": _git_sha(),
                "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
            },
            "feature_schema": FEATURE_SCHEMA,
            "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
                            "note": "threshold tuned train-side on the full corpus; "
                                    "honest generalization = the LOVO eval below"},
            "training_lineage": {
                "corpus_manifest": manifest_path,
                "split_protocol": "leave-one-video-out by match",
                "label_source": "human (rally-annotator golden CSVs)",
                "videos": [{"name": s["name"], "labels": s["labels"],
                            "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
                            "fps": s["fps"]} for s in specs],
            },
            "eval": {
                "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
                "per_video": eval_res["per_video"],
                "mean_f1": eval_res["mean_f1"],
                "floor": floor,
            },
            "attestation": {
                "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
                "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
                "no_paid_llm_training_labels": True,
                "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
            },
            "gate": verdict,
        }
        manifest_out = os.path.join(bundle_dir, "manifest.json")
        with open(manifest_out, "w", encoding="utf-8") as f:
            json.dump(manifest, f, indent=2)

        print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
        print(f"weights : {weights_path}")
        print(f"manifest: {manifest_out}")
        print(f"GATE    : ship={verdict['ship']}  floor_passed={verdict['floor_passed']}  "
              f"sign_test={verdict['sign_test']}")
        for b in verdict["blockers"]:
            print(f"  - {b}")
        return manifest


    def main() -> None:
        ap = argparse.ArgumentParser(description=__doc__)
        ap.add_argument("--manifest", required=True,
                        help="corpus manifest JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
        ap.add_argument("--floor", default=None,
                        help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
        ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
        ap.add_argument("--version", required=True, help="semver for the artifact bundle")
        args = ap.parse_args()
        run(args.manifest, args.out, args.version, floor_path=args.floor)


    if __name__ == "__main__":
        main()

```

16. assistant (2026-06-18T01:28:16.200Z)

Now let me investigate a few specific concerns. Let me look at the `gate_verdict` function's `sign_test` p-value computation carefully, and the `phase3_eval` skip-counting logic for `--only`. First, let me check the `--only` filter behavior in phase3 ? it accesses `e["video_id"]` directly which would crash if a manifest entry lacks that key. Let me check what `default_stages_for` does with an unknown segmenter and the `evaluate` contract.

Let me focus on the upload_part overflow check, which looks suspicious ? when overflow happens after writing some chunks, the file may have partial data, but `_abort_upload` removes it, so that's fine. Let me look more carefully at the `process_video` 202 endpoint and `_drain_due_jobs` for the `db_instance` global being None at import. Let me check tests to understand contracts.

17. user (2026-06-18T01:28:18.810Z)

```
66:def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
92:def report_to_dict(report: EvalReport) -> dict:
108:def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
```

18. user (2026-06-18T01:28:20.122Z)

```
__init__.py
ablation.py
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
golden_fixtures.py
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
promotion.py
rally_seg_eval.py
rally_seq_proto.py
regression.py
score_calibration.py
segmentation_metrics.py
serve_contrast.py
served_gate_a.py
splits.py
tolerance_metrics.py
training.py
windowing.py
---TESTS---
tests/test_async_jobs.py
tests/test_eval_batch.py
tests/test_eval_harness.py
tests/test_eval_training.py
tests/test_job_self_scheduling.py
```

```
tests/test_job_worker_loop.py
tests/test_training_gen0.py
tests/test_upload_download.py
```

19. assistant (2026-06-18T01:28:23.793Z)

```
Let me look at the sign_test more carefully and the test for it, plus the phase3 eval skipped
tracking. The `sign_test` p-value computation: `p = sum(comb(n, k) for k in range(wins, n + 1))
/ (2 ** n)`. This is a one-sided sign test p-value (probability of >= wins successes under
p=0.5). That looks correct. Let me look at the upload tests and async job tests to verify
contracts, and re-examine the `upload_part` overflow logic for a subtle bug.
```

20. user (2026-06-18T01:28:24.258Z)

```
1       """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3       Contract under test:
4       - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5       - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort
(413)
6       - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
7       - DELETE /api/upload/{id} ? abort removes partial state
8       - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
9       """
10      import os
11
12      import pytest
13
14      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
15      from fastapi.testclient import TestClient
16
17      import backend.main as m
18      from backend.infrastructure.database import Database
19      from backend.config.models import AppConfig
20      from backend.utils.hashing import compute_video_id
21
22
23      @pytest.fixture
24      def env(tmp_path, monkeypatch):
25          db = Database(str(tmp_path / "t.db"))
26          cfg = AppConfig()
27          cfg.output.output_dir = str(tmp_path / "out")
28          cfg.security.upload_dir = str(tmp_path / "up")
29          os.makedirs(cfg.output.output_dir, exist_ok=True)
30          monkeypatch.setattr(m, "db_instance", db)
31          monkeypatch.setattr(m, "global_config", cfg)
32          monkeypatch.setattr(m, "_uploads", {})  # isolate upload state per test
33          client = TestClient(m.app)
34          return client, db, tmp_path, cfg
35
36
37      def _init(client, filename="match.mp4", total=None):
38          body = {"filename": filename}
39          if total is not None:
40              body["total_size_bytes"] = total
41          return client.post("/api/upload/init", json=body)
42
43
44      # --- init ------------------------------------------------------------------------
45
46      def test_init_rejects_bad_extension(env):
47          client = env[0]
48          r = _init(client, "malware.exe")
49          assert r.status_code == 400
50          assert "not allowed" in r.json()["detail"]
```

```
51
52
53      def test_init_rejects_traversal_filename(env):
54          client = env[0]
55          assert _init(client, "../evil.mp4").status_code == 400
56          assert _init(client, "a/b.mp4").status_code == 400
57
58
59      def test_init_rejects_oversize_declared_total(env):
60          client, db, tmp_path, cfg = env
61          cfg.security.max_upload_mb = 1
62          r = _init(client, total=2 * 1024 * 1024)
63          assert r.status_code == 413
64
65
66      # --- full lifecycle ----------------------------------------------------------
67
68      def test_roundtrip_assembles_parts_and_matches_md5(env):
69          client, db, tmp_path, cfg = env
70          up = _init(client).json()
71          parts = [b"a" * 1000, b"b" * 2000, b"c" * 500]
72          for i, blob in enumerate(parts):
73              r = client.put(f"/api/upload/{up['upload_id']}/part/{i}", content=blob)
74              assert r.status_code == 200
75              assert r.json()["received_parts"] == i + 1
76          done = client.post(f"/api/upload/{up['upload_id']}/complete", json={"total_parts":
3})
77          assert done.status_code == 200
78          body = done.json()
79          assert body["size_bytes"] == 3500
80          with open(body["path"], "rb") as fh:
81              assert fh.read() == b"".join(parts)
82          assert body["video_id"] == compute_video_id(body["path"])
83          # partial file cleaned up, registry entry gone
84          assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]
85          assert client.put(f"/api/upload/{up['upload_id']}/part/3", content=b"x").status_code
== 404
86
87
88      def test_same_filename_twice_does_not_overwrite(env):
89          client = env[0]
90          paths = []
91          for content in (b"first", b"second"):
92              up = _init(client).json()
93              client.put(f"/api/upload/{up['upload_id']}/part/0", content=content)
94              done = client.post(f"/api/upload/{up['upload_id']}/complete",
json={"total_parts": 1})
95              paths.append(done.json()["path"])
96          assert paths[0] != paths[1]
97          with open(paths[0], "rb") as fh:
98              assert fh.read() == b"first"  # first upload untouched
99          with open(paths[1], "rb") as fh:
100             assert fh.read() == b"second"
101
102
103     # --- sequencing & caps -------------------------------------------------------
104
105     def test_out_of_order_part_is_409(env):
106         client = env[0]
107         up = _init(client).json()
108         assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 409
109
110
```

```
    def test_unknown_upload_id_is_404(env):
        client = env[0]
        assert client.put("/api/upload/nope/part/0", content=b"x").status_code == 404
        assert client.post("/api/upload/nope/complete", json={"total_parts": 1}).status_code
== 404
        assert client.delete("/api/upload/nope").status_code == 404


    def test_complete_with_wrong_part_count_is_409(env):
        client = env[0]
        up = _init(client).json()
        client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
        assert client.post(
            f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 2}).status_code
== 409
        # zero-part complete is also a 409
        up2 = _init(client).json()
        assert client.post(
            f"/api/upload/{up2['upload_id']}/complete", json={"total_parts": 0}).status_code
== 409


    def test_oversize_part_aborts_upload(env):
        client, db, tmp_path, cfg = env
        cfg.security.max_part_mb = 1
        up = _init(client).json()
        r = client.put(f"/api/upload/{up['upload_id']}/part/0",
                       content=b"x" * (1024 * 1024 + 10))
        assert r.status_code == 413
        # aborted: id unknown afterwards, no partials on disk
        assert client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x").status_code
== 404
        assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]


    def test_total_cap_across_parts_aborts_upload(env):
        client, db, tmp_path, cfg = env
        cfg.security.max_upload_mb = 1
        up = _init(client).json()
        ok = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x" * (700 *
1024))
        assert ok.status_code == 200
        r = client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x" * (700 * 1024))
        assert r.status_code == 413


    def test_abort_endpoint_removes_state(env):
        client, db, tmp_path, cfg = env
        up = _init(client).json()
        client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
        assert client.delete(f"/api/upload/{up['upload_id']}").status_code == 200
        assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 404
        assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]


    # --- download (B3)
-----------------------------------------------------------------

    def test_download_unknown_video_404(env):
        client = env[0]
        assert client.get("/api/highlights/nope/reel.mp4").status_code == 404

```

```
167
168     def test_download_traversal_filename_rejected(env):
169         client, db, tmp_path, cfg = env
170         db.add_video("vid", "/x.mp4", "processed", [])
171         r = client.get("/api/highlights/vid/..%2Fevil.mp4")
172         assert r.status_code in (400, 404)  # 400 from safe-name; 404 if the router
normalizes
173         r2 = client.get("/api/highlights/vid/a%5Cb.mp4")  # backslash separator
174         assert r2.status_code == 400
175
176
177     def test_download_missing_file_404(env):
178         client, db, tmp_path, cfg = env
179         db.add_video("vid", "/x.mp4", "processed", [])
180         assert client.get("/api/highlights/vid/reel.mp4").status_code == 404
181
182
183     def test_download_streams_compiled_file(env):
184         client, db, tmp_path, cfg = env
185         db.add_video("vid", "/x.mp4", "processed", [])
186         blob = b"\x00\x00fake-mp4-bytes" * 100
187         with open(os.path.join(cfg.output.output_dir, "reel.mp4"), "wb") as fh:
188             fh.write(blob)
189         r = client.get("/api/highlights/vid/reel.mp4")
190         assert r.status_code == 200
191         assert r.content == blob
192         assert r.headers["content-type"].startswith("video/mp4")
193         assert "reel.mp4" in r.headers.get("content-disposition", "")
194
```

21. assistant (2026-06-18T01:28:33.049Z)

Now let me carefully examine the `upload_part` total-cap logic. The check is `st["bytes"] + received > max_bytes`. The test `test_total_cap_across_parts_aborts_upload` uses `max_upload_mb=1` (= 1048576 bytes). Part 0 is 700*1024 = 716800 bytes, accepted. Part 1 is also 716800 bytes; `st["bytes"]` is now 716800, and during streaming `received` accumulates. When `716800 + received > 1048576`, i.e. `received > 331776`, it aborts. Correct.

Now let me look at a subtle issue. In `upload_part`, after the loop on overflow, the partial bytes that were written before the break are NOT trimmed ? but `_abort_upload` removes the whole file, so it's fine. Let me look at the async job tests and the `process_video` endpoint's `db_instance` usage. Let me focus on `default_stages_for` for unknown segmenters and the `evaluate` stages contract. But first let me re-examine `aggregate` micro_f1 ? it's computed via `_safe_div(2*p*r, p+r)`, fine.

Let me look at the gen0 training test and check `sign_test`/`gate_verdict` contract more carefully, plus `train_final` threshold selection.

22. user (2026-06-18T01:28:34.102Z)

```
1       """Gen-0 harness (training/gen0): end-to-end on SYNTHETIC fixtures only.
2
3       Real trajectories/labels are never committed (training/README.md rule 4 ? consent ledger
4       pending). The synthetic corpus mirrors tests/test_rally_seq_proto.py: separable rally vs
5       dead-time signal, three rallies per 'video'.
6       """
7       import csv
8       import json
9
10      import numpy as np
11      import pytest
12
13      from training.gen0 import harness
14
15
```

```
    def _write_traj(path, fps=10.0):
        rows = [("Frame", "Visibility", "X", "Y")]
        n = int(120 * fps)
        for f in range(n):
            t = f / fps
            in_rally = (t < 20) or (40 <= t < 60) or (80 <= t < 100)
            if in_rally:
                x = 50 + 45 * np.sin(2 * np.pi * (f % 10) / 10.0)
                rows.append((f, 1, round(float(x), 2), 50))
            else:
                rows.append((f, 1, 50.0, 50) if f % 5 == 0 else (f, 0, -1, -1))
        with open(path, "w", newline="") as fh:
            csv.writer(fh).writerows(rows)
        return str(path)


    def _write_labels(path):
        with open(path, "w", newline="", encoding="utf-8") as fh:
            w = csv.writer(fh)
            w.writerow(["rally_number", "start_time", "end_time"])
            w.writerow([1, 0.0, 20.0])
            w.writerow([2, 40.0, 60.0])
            w.writerow([3, 80.0, 100.0])
        return str(path)


    @pytest.fixture
    def corpus(tmp_path):
        specs = [{
            "name": f"v{i}", "trajectory": _write_traj(tmp_path / f"t{i}.csv"),
            "fps": 10.0, "frame_width": 100.0, "labels": _write_labels(tmp_path /
f"g{i}.csv"),
        } for i in range(2)]
        mpath = tmp_path / "manifest.json"
        mpath.write_text(json.dumps(specs), encoding="utf-8")
        return str(mpath), tmp_path


    def test_harness_end_to_end_emits_artifact_bundle(corpus):
        mpath, tmp_path = corpus
        floor = tmp_path / "floor.json"
        floor.write_text(json.dumps({
            "name": "synthetic-floor", "mean_f1": 0.10,
            "per_video": {"v0": 0.10, "v1": 0.05},
        }), encoding="utf-8")

        out = harness.run(mpath, str(tmp_path / "artifacts"), "0.0.1",
floor_path=str(floor))

        bundle = tmp_path / "artifacts" / "0.0.1"
        assert (bundle / "weights.json").exists()
        assert (bundle / "manifest.json").exists()

        # Manifest contract (the reviewable surface).
        m = json.loads((bundle / "manifest.json").read_text(encoding="utf-8"))
        assert m["artifact"]["name"] == "rally_tal_gen0"
        assert m["artifact"]["weights_sha256"]
        assert m["feature_schema"]["version"] >= 1
        assert m["feature_schema"]["fps_semantics"] == "per-second"
        assert 0.2 <= m["calibration"]["threshold"] <= 0.8
        assert len(m["training_lineage"]["videos"]) == 2
        assert all(v["gt_hash"] for v in m["training_lineage"]["videos"])
        assert m["eval"]["mean_f1"] > 0.5            # separable synthetic signal
        assert m["attestation"]["no_paid_llm_training_labels"] is True

```

```
79          # Gate: floor beaten, but NEVER self-ships (provisional target + C3 open; see #172).
80          g = m["gate"]
81          assert g["ship"] is False
82          assert g["floor_passed"] is True
83          assert any("C3" in b for b in g["blockers"])
84          assert any("NOT YET CALIBRATED" in b for b in g["blockers"])
85          assert any("#172" in b for b in g["blockers"])
86          assert out["gate"] == g
87
88
89      def test_weights_round_trip_and_schema_pin(corpus):
90          mpath, tmp_path = corpus
91          harness.run(mpath, str(tmp_path / "artifacts"), "0.0.2", floor_path=None)
92          payload = json.loads((tmp_path / "artifacts" / "0.0.2" /
"weights.json").read_text())
93          from backend.eval.classifier import LogisticRegression
94          from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA_VERSION
95          clf = LogisticRegression.from_dict(payload)
96          assert payload["feature_schema_version"] == FEATURE_SCHEMA_VERSION
97          assert list(clf.feature_names) == FEAT_NAMES
98          probs = clf.predict_proba(np.zeros((4, len(FEAT_NAMES))))
99          assert probs.shape == (4,)
100
101
102     def test_sign_test_math():
103         # 5/6 wins -> one-sided p = 7/64 ~ 0.109 (NOT significant ? honesty pinned).
104         wins, n, p = harness.sign_test([1, 1, 1, 1, 1, 0], [0, 0, 0, 0, 0, 1])
105         assert (wins, n) == (5, 6)
106         assert p == pytest.approx(7 / 64)
107         # 6/6 -> p = 1/64 < 0.05.
108         _, _, p6 = harness.sign_test([1] * 6, [0] * 6)
109         assert p6 == pytest.approx(1 / 64)
110
```

23. assistant (2026-06-18T01:28:38.986Z)

The sign_test math is verified by tests. Now let me dig into specific potential bugs. Let me look at `run_phase3_eval.py` line 138 ? `db.query_play_segments(video_id, {}) or db.query_candidate_segments(video_id)`. And the `--only` filter at line 110 accessing `e["video_id"]` and line 121 `vid = e["video_id"]`. These would KeyError on a malformed manifest entry, but that's the manifest's contract.

Let me focus on a real concern: in `run_phase3_eval.py`, the segmenter `process_video` returns `segs` which could be a `Failure` object (not a list), and line 146 does `len(segs)`. Let me check what `process_video` returns and whether Failure has `__len__`.

24. user (2026-06-18T01:28:41.467Z)

```
27:class Success(Generic[T]):
42:class Failure:
```

25. user (2026-06-18T01:28:42.154Z)

```
60              rows = db.query_play_segments(video_id, {})
61          else:
62              raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63          return _rows_to_intervals(rows)
64
65
66      def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67                   stages: List[str], iou_threshold: float = 0.5,
68                   detector: Optional[str] = None,
69                   with_pr_curve: bool = False) -> EvalReport:
70          """Score the requested stages for one video against ground truth."""
```

```
71          report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72                              gt_intervals=gt_intervals)
73          for stage in stages:
74              preds = get_predictions(db, video_id, stage, detector=detector)
75              mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76              s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77              curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78              report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                                 score=s, match_result=mr, pr_curve=curve)
80          return report
81
82
83      # ---------------------------------------------------------------------- reporting
84
85      def _fmt_ts(seconds: float) -> str:
86          seconds = max(0, int(seconds))
87          h, rem = divmod(seconds, 3600)
88          m, s = divmod(rem, 60)
89          return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92      def report_to_dict(report: EvalReport) -> dict:
93          """JSON-serialisable view of an EvalReport."""
94          out: Dict[str, Any] = {
95              "video_id": report.video_id,
96              "iou_threshold": report.iou_threshold,
97              "num_ground_truth": len(report.gt_intervals),
98              "stages": {},
99          }
100         for stage, sr in report.stages.items():
101             out["stages"][stage] = {
102                 "score": sr.score.as_dict(),
103                 "pr_curve": [s.as_dict() for s in sr.pr_curve],
104             }
105         return out
106
107
108     def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109         """Human-readable console report for one video."""
110         lines = [
111             "=" * 64,
112             f" Rally evaluation ? video '{report.video_id}'",
113             f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114             "=" * 64,
115             f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116             f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117             "-" * 64,
118         ]
119         for stage in report.stages:
```

26. assistant (2026-06-18T01:28:45.266Z)

Let me check whether `process_video` can return a Failure and whether `len()` on it would crash in run_phase3_eval line 146. Let me check Failure's interface and the segmenter base `process_video` return contract.

27. user (2026-06-18T01:28:47.007Z)

```
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Union
22
23      T = TypeVar("T")
24
25
```

```
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
```

28. assistant (2026-06-18T01:28:50.685Z)

`Failure` has no `__len__`, so in `run_phase3_eval.py` line 146 `len(segs)` would crash if `process_video` returns a `Failure`. But this script appears to assume `process_video` returns a list directly. Let me check the actual return type of `process_video` in the segmenter base, and whether it returns a raw list or a Result.

29. user (2026-06-18T01:28:52.174Z)

```
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py:35:    def process_video(self,
video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) ->
List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-36-        """
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-37-        Processes a video to detect play segments
using Google Gemini Multimodal Video API,
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-38-        populates SQLite, and indexes detailed
events.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-39-        """
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-40-        # Get sport profile config
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-41-        sport_name = self.config.get("sport",
"badminton")
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py-42-        try:
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\gemini.py-43-            sport_profile =
sport_registry.get(sport_name)
--
.claude\worktrees\hunt-base\backend\pipeline\segmenters\motion.py:20:    def process_video(self,
video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) ->
List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-21-        """
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-22-        Processes a video to detect play segments
using motion heuristics,
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-23-        populates SQLite with detected segments,
and indexes detailed events.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-24-        """
.claude\worktrees\hunt-base\backend\pipeline\segmenters\motion.py-25-        if not player_pool:
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-26-            player_pool = ["Avi", "Suresh",
"Ramesh", "Deepak"]
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-27-
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\motion.py-28-        cap = cv2.VideoCapture(video_path)
--
.claude\worktrees\hunt-base\backend\pipeline\segmenters\base.py:24:    def process_video(self,
video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) ->
"Result[List[Dict[str, Any]]]":
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-25-        """
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-26-        Processes a video to detect play segments,
populates SQLite database,
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-27-        and returns a Result.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-28-
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-29-        Use isinstance(result, Failure) (or
result.is_success()) to distinguish
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-30-        "no segments found" from "something went
wrong" (missing key, healthcheck
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-31-        failure, etc.). This is the start of the
standardized error/result contract.
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\base.py-32-        """
--
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:106:    def
process_video(self, video_id: str, video_path: str,  # type: ignore[override]
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py-107-
player_pool: Optional[List[str]] = None,
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py-108-
max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py-109-        #
override-ignore: the ABC declares the Result contract (Success|Failure)
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\trajectory_hybrid.py-110-        # but every live segmenter
still returns a bare list ? the documented
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\trajectory_hybrid.py-111-        # deliberate-incremental gap
(CODE_MAP gotchas; main.py handles both).
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
```

```
base\backend\pipeline\segmenters\trajectory_hybrid.py-112-        label = self.display_label
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\trajectory_hybrid.py-113-        # --- Sport profile + AI
provider wiring (identical across segmenters) ---
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\trajectory_hybrid.py-114-        sport_name =
self.config.get("sport", "badminton")
--
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:57:    def
process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames:
int = None) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\yolo_hybrid.py-58-        self.person.lazy_load_model()
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\yolo_hybrid.py-59-
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\yolo_hybrid.py-60-        # Get sport profile config
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\yolo_hybrid.py-61-        sport_name = self.config.get("sport",
"badminton")
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py-62-        try:
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\yolo_hybrid.py-63-            sport_profile =
sport_registry.get(sport_name)
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py-64-        except
KeyError as e:
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\pipeline\segmenters\yolo_hybrid.py-65-            logger.error(str(e))
```

30. assistant (2026-06-18T01:28:58.380Z)

Live segmenters return a bare list (the documented gap; `main.py` handles both via `result.value if hasattr(result, "value")`). In `run_phase3_eval.py`, `segs` is used as a list directly which matches the live-segmenter reality. The Failure path is theoretically possible but `main.py` explicitly guards it while the script doesn't ? however since live segmenters never return Failure, this isn't a reachable defect. I'll set this aside as not concrete enough.

Let me now look more carefully at potential real bugs. Let me examine `get_predictions` for the candidates stage and `default_stages_for` with `tracknet_*`. Also let me check the `process_video` endpoint validation more closely ? specifically `validate_input_path` and the `db_instance.create_job` signature. Let me check the async jobs tests and DB job methods.

31. user (2026-06-18T01:28:59.229Z)

```
1       """Async job model (DEFERRED_HARDENING_PLAN #16).
2
3       Contract under test:
4       - POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
5       - GET /api/jobs/{id} ? {status, progress, result, reports, ...}
6       - job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
7         verdict (e.g. failed validation) lives in result["status"].
8       - worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job
failed
9       - startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
10      - migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
11      """
12      import sqlite3
13      import time
14      from concurrent.futures import ThreadPoolExecutor
15
16      import pytest
17
18      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
19      from fastapi.testclient import TestClient
20
```

```
    import backend.main as m
    from backend.infrastructure.database import Database
    from backend.config.models import AppConfig
    from backend.pipeline.validators.base import ValidationResult


    class _InlineExecutor:
        """submit() runs the fn synchronously ? deterministic tests, no threads."""

        def submit(self, fn, *args, **kwargs):
            fn(*args, **kwargs)


    @pytest.fixture
    def env(tmp_path, monkeypatch):
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        client = TestClient(m.app)
        return client, db, tmp_path, monkeypatch


    def _video(tmp_path, name="m.mp4"):
        p = tmp_path / name
        p.write_bytes(b"not-a-real-video-but-hashable")
        return p


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    def _fail():
        return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]


    class _FakeSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
            return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]


    class _RaisingSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            self.db.add_play_segment(video_id, 9.0, 10.0)  # half-written row pre-crash
            raise RuntimeError("boom")


    # --- migration ---------------------------------------------------------

    def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
        db = Database(str(tmp_path / "fresh.db"))
        with db._get_connection() as conn:
            # v5 = per-user model store (PERSONALIZATION_PLAN.md §2); the jobs table arrived
at v2/v3.
```

```
            assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
            cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
        assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <=
cols
        assert {"policy", "next_poll_at"} <= cols           # v3 self-scheduling columns


    def test_migration_upgrades_v1_db_in_place(tmp_path):
        path = str(tmp_path / "v1.db")
        conn = sqlite3.connect(path)
        conn.execute(
            "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
            "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
        # A genuine v1 DB also has candidate_segments (created in the version<1 block);
include it
        # so the v4 personalization ALTER (which adds user_id to that table) has a table to
alter.
        conn.execute(
            "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
            "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
            "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)")
        conn.execute("PRAGMA user_version = 1")
        conn.commit()
        conn.close()

        db = Database(path)  # ladder should take it 1 ? 5
        with db._get_connection() as c:
            assert c.execute("PRAGMA user_version").fetchone()[0] == 5
            assert c.execute(
                "SELECT name FROM sqlite_master WHERE type='table' AND
name='jobs'").fetchone()
            # Restore v5 personalization coverage (user_models table + user_id on
candidate_segments).
            # These were dropped in prior edit; add (do not replace) per review.
            tables = {r[0] for r in c.execute(
                "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
            assert "user_models" in tables
            cand_cols = {r["name"] for r in c.execute(
                "PRAGMA table_info(candidate_segments)").fetchall()}
            assert "user_id" in cand_cols


    def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
        """Item 5 positive: direct CLI-style path (m._execute_processing) vs job-worker path
        (submit + _InlineExecutor leading to _run_claimed_job / drain, plus explicit
_run_claimed_job)
        produce equivalent outcomes for the same ProcessRequest shape. Exercises success,
        Failure, and empty-run branches. Real DB queries using vid (no self-compares or
tautologies).
        """
        client, db, tpath, mp = env
        vid_file = _video(tpath)
        vid = m.compute_video_id(str(vid_file))
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        req = m.ProcessRequest(video_path=str(vid_file), segmenter="x")

        # Direct CLI-style path
        direct_out = m._execute_processing(req)
        segs = db.query_play_segments(vid, {})
        assert len(segs) >= 1, f"direct path must affect segments for vid {vid}"
        assert direct_out.get("result", {}).get("status") in (None, "success", "processed")
or "play_segments" in str(direct_out)

```

```
        # Job-worker path via public API submit (exercises job creation + inline executor +
internal _run_claimed_job)
        rj = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
        job_out = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
        segs_j = db.query_play_segments(vid, {})
        assert job_out["result"]["status"] == "success"
        assert len(segs_j) >= 1

        # Explicit job-worker via _run_claimed_job (create + mark running + drive the
claimed runner)
        vid_file2 = _video(tpath, name="m2.mp4")
        vid2 = m.compute_video_id(str(vid_file2))
        jid = f"parity-{vid2[:8]}"
        db.create_job(jid, str(vid_file2))
        db.mark_job_running(jid)
        job_row = db.get_job(jid)
        m._run_claimed_job(job_row)
        segs2 = db.query_play_segments(vid2, {})
        assert len(segs2) >= 1, f"_run_claimed_job path must produce segments for vid
{vid2}"

        # Failure branch (direct + job submit)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
        req_f = m.ProcessRequest(video_path=str(vid_file))
        direct_f = m._execute_processing(req_f)
        rf = client.post("/api/process", json={"video_path": str(vid_file)})
        jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
        assert direct_f.get("result", {}).get("status") == "failed" or
jobf["result"]["status"] == "failed"

        # Empty-run branch (success verdict but zero play_segments from this execution)
        class _EmptySeg:
            def __init__(self, db, cfg):
                self.db = db
            def process_video(self, video_id, video_path, *a, **k):
                return []
        mp.setattr(m.segmenter_registry, "get", lambda name: _EmptySeg)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        req_e = m.ProcessRequest(video_path=str(vid_file))
        de = m._execute_processing(req_e)
        re = client.post("/api/process", json={"video_path": str(vid_file)})
        je = client.get(f"/api/jobs/{re.json()['job_id']}").json()
        assert de.get("result", {}).get("status") in (None, "success")
        assert je["result"]["status"] == "success"
        # (reingest safety for prior on empty/fail covered in dedicated tests)


    # --- submit-time fast-fail ------------------------------------------------------

    def test_submit_returns_202_with_job_id(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
        r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
        assert r.status_code == 202
        body = r.json()
        assert body["job_id"]
        assert body["status"] == "queued"
        assert body["poll"].endswith(body["job_id"])


    def test_submit_rejects_null_byte_path(env):
```

```
        client = env[0]
        r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})
        assert r.status_code == 400


    def test_submit_missing_file_is_404(env):
        client, db, tmp_path, mp = env
        r = client.post("/api/process", json={"video_path": str(tmp_path / "nope.mp4")})
        assert r.status_code == 404


    def test_submit_unknown_segmenter_400_and_no_job_row(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.segmenter_registry, "get", lambda name: None)
        mp.setattr(m.segmenter_registry, "list_available", lambda: {"motion": "..."})
        r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"nope"})
        assert r.status_code == 400
        assert "Available segmenters" in r.json()["detail"]
        with db._get_connection() as c:
            assert c.execute("SELECT COUNT(*) FROM jobs").fetchone()[0] == 0


    # --- end-to-end job lifecycle (inline executor = deterministic) ----------------------

    def test_job_done_with_segments_and_video_id(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        job_id = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
        r = client.get(f"/api/jobs/{job_id}")
        assert r.status_code == 200
        body = r.json()
        assert body["status"] == "done"
        assert body["progress"] == "finished"
        assert body["error"] is None
        assert body["result"]["status"] == "success"
        assert len(body["result"]["play_segments"]) == 1
        assert "reports" in body  # None without obsreport; key must exist per the contract

        vid = m.compute_video_id(str(vid_file))
        assert body["video_id"] == vid
        assert len(db.query_play_segments(vid, {})) == 1
        assert db.get_video(vid)["status"] == "processed"


    def test_job_done_but_pipeline_verdict_failed_preserves_prior(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        vid = m.compute_video_id(str(vid_file))
        db.add_video(vid, str(vid_file), "processed", [])
        db.add_play_segment(vid, 1.0, 2.0)  # prior good result
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))

        job_id = client.post("/api/process", json={"video_path":
str(vid_file)}).json()["job_id"]
        body = client.get(f"/api/jobs/{job_id}").json()
        # Worker finished fine ? the PIPELINE verdict is the failure.
        assert body["status"] == "done"
```

```
        assert body["result"]["status"] == "failed"
        # PR-C semantics survive the async move: prior segments intact.
        assert len(db.query_play_segments(vid, {})) == 1


    def test_worker_crash_marks_failed_and_restores_prior_segments(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        vid = m.compute_video_id(str(vid_file))
        db.add_video(vid, str(vid_file), "processed", [])
        db.add_play_segment(vid, 1.0, 2.0)  # prior good result
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _RaisingSeg)

        job_id = client.post(
            "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
        body = client.get(f"/api/jobs/{job_id}").json()
        assert body["status"] == "failed"
        assert "RuntimeError" in body["error"]
        assert body["result"] is None
        # The half-written 9.0?10.0 row was pruned; the prior 1.0?2.0 row survives.
        segs = db.query_play_segments(vid, {})
        assert len(segs) == 1 and segs[0]["start_time"] == 1.0


    def test_progress_stages_are_recorded(env):
        client, db, tmp_path, mp = env
        vid_file = _video(tmp_path)
        mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
        mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)

        stages = []
        real = db.set_job_progress

        def spy(job_id, progress):
            stages.append(progress)
            return real(job_id, progress)

        mp.setattr(db, "set_job_progress", spy)
        client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
        assert "hashing" in stages
        assert any(s.startswith("segmenting") for s in stages)


    # --- polling endpoint ------------------------------------------------------

    def test_get_unknown_job_is_404(env):
        client = env[0]
        assert client.get("/api/jobs/deadbeef").status_code == 404


    # --- crash recovery --------------------------------------------------------

    def test_fail_stale_jobs_terminalizes_queued_and_running_only(env):
        client, db, tmp_path, mp = env
        db.create_job("q1", "/v.mp4")
        db.create_job("r1", "/v.mp4")
        db.mark_job_running("r1")
        db.create_job("d1", "/v.mp4")
        db.mark_job_done("d1", {"status": "success"})

        swept = db.fail_stale_jobs("interrupted by server restart")
        assert swept == 2
        assert db.get_job("q1")["status"] == "failed"
```

```
325         assert db.get_job("r1")["status"] == "failed"
326         assert "restart" in db.get_job("q1")["error"]
327         assert db.get_job("d1")["status"] == "done"  # terminal states untouched
328
329
330     # --- real executor (cross-thread SQLite/WAL sanity) ---------------------------------
331
332     def test_real_executor_completes_job(env):
333         client, db, tmp_path, mp = env
334         vid_file = _video(tmp_path)
335         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
336         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
337         pool = ThreadPoolExecutor(max_workers=1)
338         mp.setattr(m, "_get_executor", lambda: pool)
339         try:
340             job_id = client.post(
341                 "/api/process", json={"video_path": str(vid_file), "segmenter":
"x"}).json()["job_id"]
342             deadline = time.time() + 10.0
343             body = None
344             while time.time() < deadline:
345                 body = client.get(f"/api/jobs/{job_id}").json()
346                 if body["status"] in ("done", "failed"):
347                     break
348                 time.sleep(0.05)
349             assert body is not None and body["status"] == "done"
350             assert body["result"]["status"] == "success"
351         finally:
352             pool.shutdown(wait=True)
353
```

32. assistant (2026-06-18T01:29:06.366Z)

The async job flow is well-tested. Now let me look at the `create_job` signature to verify the call in `process_video` (line 370-371) matches. Let me also check `download_highlight` ? there's a subtle ordering issue: it calls `safe_output_filename` first, then checks the video record. Let me also examine `_finalize_reingest` and the `db_instance` global being None at startup. Let me check the `create_job` and `get_video` DB methods.

33. user (2026-06-18T01:29:06.795Z)

```
270:    def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
271-                       play_after: Optional[int] = None, cand_upto: Optional[int] = None,
272-                       cand_after: Optional[int] = None) -> None:
273-        """Delete play/candidate segments by id range (events cascade from play_segments).
274-
--
355:    def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
356-                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
357-        """Fetch candidate detections for the annotation / fine-tuning workflow.
358-        `stats` and `detection_params` are deserialized from JSON."""
359-        query = "SELECT * FROM candidate_segments WHERE video_id = ?"
--
380:    def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
381-        with self._connect() as conn:
382-            row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
383-            if row:
384-                d = dict(row)
--
389:    def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
390-        """
391-        Dynamically queries play segments based on filters:
392-        - duration_min: float
```

```
393-        - server: string
--
448:    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449-                   player_pool: Optional[List[str]] = None,
450-                   max_frames: Optional[int] = None,
451-                   policy: Optional[Dict[str, Any]] = None) -> bool:
452-        """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
```

34. assistant (2026-06-18T01:29:09.693Z)

The `create_job` signature matches the call. Now let me look carefully at a potential bug in the `query_play_segments` filter handling in `/api/query`. When `duration_min` is None or other filters are None, they're passed in the filters dict. Let me check how `query_play_segments` handles None filter values.

35. user (2026-06-18T01:29:10.686Z)

```
389         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
390             """
391             Dynamically queries play segments based on filters:
392             - duration_min: float
393             - server: string
394             - receiver: string
395             - event_type: string (e.g. smash, foul)
396             """
397             query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
398             params = [video_id]
399
400             if filters.get("duration_min"):
401                 query += " AND r.duration >= ?"
402                 params.append(filters["duration_min"])
403
404             if filters.get("server"):
405                 query += " AND r.server = ?"
406                 params.append(filters["server"])
407
408             if filters.get("receiver"):
409                 query += " AND r.receiver = ?"
410                 params.append(filters["receiver"])
411
412             if filters.get("event_type"):
413                 # Check if there is an event of a specific type inside this segment
414                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
415                 params.append(filters["event_type"])
416
417             segments_list = []
418             with self._connect() as conn:
419                 cursor = conn.cursor()
420                 rows = cursor.execute(query, params).fetchall()
421
422                 for row in rows:
423                     r_dict = dict(row)
424                     r_id = r_dict["id"]
425                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
426
427                     # Fetch associated events
428                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
429                     r_dict["events"] = []
430                     for er in event_rows:
431                         e_dict = dict(er)
432                         e_dict["details"] = json.loads(e_dict["details"] or "{}")
```

```
433                         r_dict["events"].append(e_dict)
434
435                     segments_list.append(r_dict)
436
437             return segments_list
438
439         def clear_video_data(self, video_id: str):
440             with self._connect() as conn:
441                 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442                 conn.commit()
443
444         # --- Async job store (DEFERRED_HARDENING_PLAN #16) -------------------------------
445         # Writers follow the repo convention (swallow + logger.error, return bool); the
446         # worker logs loudly on a failed transition so a job can't silently wedge.
447
448         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
```

36. assistant (2026-06-18T01:29:23.269Z)

Query handling is safe (uses `.get()` with truthiness). Now let me carefully re-examine the `upload_part` total-cap logic for a subtle off-by-one / accounting bug. Look at lines 478-497:

```python
received = 0
overflow = None
with open(st["tmp_path"], "ab") as fh:
    async for chunk in request.stream():
        received += len(chunk)
        if received > part_bytes:
            overflow = ...
            break
        if st["bytes"] + received > max_bytes:
            overflow = ...
            break
        fh.write(chunk)
...
with _uploads_lock:
    st["parts"] += 1
    st["bytes"] += received
```

There's a subtle bug here. When overflow occurs, `received` has already been incremented by the overflowing chunk's length, but `fh.write(chunk)` was NOT called for that chunk. The upload is aborted via `_abort_upload`, so the file is deleted ? that's fine. But consider the non-overflow case: this is fine since every counted chunk is also written.

Now let me think about a different angle: the per-part limit check and the total accounting. Let me look at whether there's a discrepancy. Actually let me focus on `download_highlight`'s ordering and `compile_highlights` `segment_ids` matching. Let me re-look at `compile_highlights` ? `matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]`. This is fine.

Let me now examine the `nightly_regression.ps1` and `register_nightly_task.ps1` scripts and the gen0 `train_final` for a subtle issue with threshold selection when `_seg_f1` ties. Let me also re-examine `default_stages_for` ? the docstring says auto scores both for `tracknet_*` but `_FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}`. Any unknown segmenter gets both stages. That's intended.

Let me look at the PowerShell scripts and check for issues in `run_phase3_eval` around line 92 `os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)`.

37. user (2026-06-18T01:29:24.860Z)

```
1       <#
2       .SYNOPSIS
3           Register (or remove) the nightly rally-quality regression as a Windows Scheduled
```

```
Task.
4
5       .DESCRIPTION
6           Creates a daily user-level task that runs scripts/nightly_regression.ps1 against a
repo
7           checkout. The action is inline-guarded, so it's safe to register BEFORE this work is
merged
8           into the target checkout: it no-ops + logs "awaiting" until the wrapper exists
there, then
9           self-activates. Idempotent (-Force re-registers); reversible (-Unregister).
10
11          Registers under the current user (no admin / no -RunLevel Highest needed ?
consistent with the
12          repo's other scheduled tasks). Uses -StartWhenAvailable so a run missed while the
machine slept
13          is caught up on wake.
14
15      .EXAMPLE
16          # Register against the main checkout, nightly at 03:00:
17          pwsh -File scripts/register_nightly_task.ps1 -RepoRoot
'C:\Users\avidu\Projects\badminton-highlight-indexer'
18
19      .EXAMPLE
20          # Remove it:
21          pwsh -File scripts/register_nightly_task.ps1 -Unregister
22      #>
23      [CmdletBinding()]
24      param(
25          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
26          [string]$TaskName = 'KhelSutraNightlyRegression',
27          [string]$Time = '03:00',
28          [switch]$Pull,
29          [switch]$Unregister
30      )
31
32      if ($Unregister) {
33          Unregister-ScheduledTask -TaskName $TaskName -Confirm:$false -ErrorAction
SilentlyContinue
34          Write-Host "Unregistered scheduled task '$TaskName'."
35          return
36      }
37
38      $wrapper = Join-Path $RepoRoot 'scripts\nightly_regression.ps1'
39      $ps = (Get-Command pwsh.exe -ErrorAction SilentlyContinue).Source
40      if (-not $ps) { $ps = 'powershell.exe' }
41
42      # Inline-guarded action: call the wrapper if present, else log "awaiting merge" and exit
0.
43      $marker = Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt'
44      $pullArg = if ($Pull) { ' -Pull' } else { '' }
45      $inner = @"
46      `$w = '$wrapper'
47      if (Test-Path `$w) { & `$w -RepoRoot '$RepoRoot'$pullArg }
48      else {
49        New-Item -ItemType Directory -Force -Path (Split-Path -Parent '$marker') | Out-Null
50        "`$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')  status=SKIPPED exit=0  wrapper absent
(awaiting merge) at '$wrapper'" | Out-File -FilePath '$marker' -Encoding utf8
51      }
52      "@
53      $encoded = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($inner))
54
55      $action = New-ScheduledTaskAction -Execute $ps `
56          -Argument "-NoProfile -ExecutionPolicy Bypass -EncodedCommand $encoded"
57      $trigger = New-ScheduledTaskTrigger -Daily -At $Time
```

```
    $settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -DontStopOnIdleEnd `
        -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
        -ExecutionTimeLimit (New-TimeSpan -Minutes 30)
    $principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME"
-LogonType Interactive -RunLevel Limited

    Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger `
        -Settings $settings -Principal $principal -Force `
        -Description "Nightly rally-quality regression tripwire: re-scores the golden corpus
(cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline." | Out-Null

    Write-Host "Registered scheduled task '$TaskName' ? daily at $Time, RepoRoot=$RepoRoot."
    Write-Host "Wrapper: $wrapper"
    Write-Host "Run now to test:  Start-ScheduledTask -TaskName '$TaskName'"
    Write-Host "Remove:           pwsh -File scripts/register_nightly_task.ps1 -Unregister"

```

38. user (2026-06-18T01:29:25.088Z)

```
    import os

    from backend.eval import batch


    # ---- path resolution ------------------------------------------------------

    def test_resolve_relative_windows_path():
        got = batch.resolve_video_path("./data/videos\\shuttleset_1.mp4", "/collector")
        assert got == os.path.normpath("/collector/data/videos/shuttleset_1.mp4")


    def test_resolve_absolute_path_kept():
        got = batch.resolve_video_path("C:/vids/x.mp4", "/collector")
        assert got == os.path.normpath("C:/vids/x.mp4")


    def test_resolve_none():
        assert batch.resolve_video_path(None, "/collector") is None
        assert batch.resolve_video_path("", "/collector") is None


    # ---- stage selection ------------------------------------------------------

    def test_auto_stage_final_only_for_motion():
        assert batch.default_stages_for("motion", "auto") == ["final"]
        assert batch.default_stages_for("gemini", "auto") == ["final"]


    def test_auto_stage_both_for_two_stage_detector():
        assert batch.default_stages_for("yolo_hybrid", "auto") == ["candidates", "final"]


    def test_explicit_stage_overrides_auto():
        assert batch.default_stages_for("motion", "both") == ["candidates", "final"]
        assert batch.default_stages_for("yolo_hybrid", "final") == ["final"]


    # ---- aggregation ----------------------------------------------------------

    def _vid(tp, fp, fn, miou, f1):
        return {"stages": {"final": {"score": {
            "true_positives": tp, "false_positives": fp, "false_negatives": fn,
            "mean_iou": miou, "f1": f1,
        }}}}
```

```
46
47
48      def test_aggregate_micro_and_macro():
49          per_video = [
50              _vid(tp=8, fp=2, fn=2, miou=0.9, f1=0.8),   # P=.8 R=.8 F1=.8
51              _vid(tp=6, fp=0, fn=4, miou=0.8, f1=0.75),  # P=1 R=.6
52          ]
53          agg = batch.aggregate(per_video, ["final"])["final"]
54          assert agg["tp"] == 14 and agg["fp"] == 2 and agg["fn"] == 6
55          assert agg["videos"] == 2
56          # micro precision = 14/16, recall = 14/20
57          assert round(agg["micro_precision"], 3) == round(14 / 16, 3)
58          assert round(agg["micro_recall"], 3) == round(14 / 20, 3)
59          # TP-weighted mean IoU = (0.9*8 + 0.8*6)/14
60          assert round(agg["mean_iou"], 4) == round((0.9 * 8 + 0.8 * 6) / 14, 4)
61          # macro F1 = mean of per-video F1
62          assert round(agg["macro_f1"], 4) == round((0.8 + 0.75) / 2, 4)
63
64
65      def test_aggregate_handles_missing_stage_and_zero_division():
66          agg = batch.aggregate([], ["final"])["final"]
67          assert agg["tp"] == 0 and agg["micro_f1"] == 0.0 and agg["mean_iou"] == 0.0
68
```

39. assistant (2026-06-18T01:29:29.654Z)

Now let me look at `report_to_dict` more carefully. In `aggregate`, line 84: `s = sr["score"]` and accesses `s["true_positives"]`, `s["false_positives"]`, `s["false_negatives"]`, `s["mean_iou"]`, `s["f1"]`. But `report_to_dict` produces `score` via `sr.score.as_dict()`. Let me check what `score.as_dict()` produces ? does it have `f1`, `mean_iou`, `true_positives`? Let me verify the contract between `report_to_dict` (used in run_phase3_eval) and `aggregate`.

40. user (2026-06-18T01:29:32.191Z)

```
90:    true_positives: int
91:    false_positives: int
92:    false_negatives: int
96:    mean_iou: float            # over matched pairs (0.0 if none)
100:    def as_dict(self) -> dict:
123:        mean_iou = sum(m.iou for m in mr.matches) / tp
127:        mean_iou = mean_start_err = mean_end_err = 0.0
132:        true_positives=tp, false_positives=fp, false_negatives=fn,
134:        mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
```

41. user (2026-06-18T01:29:35.030Z)

```
88          num_pred: int
89          num_gt: int
90          true_positives: int
91          false_positives: int
92          false_negatives: int
93          precision: float
94          recall: float
95          f1: float
96          mean_iou: float            # over matched pairs (0.0 if none)
97          mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
98          mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds
99
100         def as_dict(self) -> dict:
101             return self.__dict__.copy()
102
103
104     def score(preds: List[Interval], gts: List[Interval],
105               iou_threshold: float = 0.5,
```

```
106               match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)
113
114         tp = len(mr.matches)
115         fp = len(mr.unmatched_pred_indices)
116         fn = len(mr.unmatched_gt_indices)
117
118         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
121
122         if mr.matches:
123             mean_iou = sum(m.iou for m in mr.matches) / tp
124             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
125             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
126         else:
127             mean_iou = mean_start_err = mean_end_err = 0.0
```

42. assistant (2026-06-18T01:29:41.781Z)

The score dict has all the keys `aggregate` needs. Contract is consistent.

Now let me look at the PowerShell `nightly_regression.ps1` wrapper for issues (it's a script, not the excluded `nightly_regression.py`). The exclusion is only for `scripts/nightly_regression.py`.

43. user (2026-06-18T01:29:41.983Z)

```
1       <#
2       .SYNOPSIS
3           Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5       .DESCRIPTION
6           Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
7           (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline.
Writes a
8           dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker,
then exits
9           with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11          GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present
yet (e.g.
12          before this PR is merged into the checkout the task points at) ? so a registered
task never
13          errors in the interim and self-activates once the files land.
14
15          Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE
the repo on
16          this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18      .PARAMETER RepoRoot
19          Repo root to test. Default: the parent of this script's directory.
20
21      .PARAMETER PythonExe
22          Python interpreter. Default: 'python'.
```

```
23
24      .PARAMETER Pull
25          If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26          that may hold a sibling session's WIP).
27      #>
28      [CmdletBinding()]
29      param(
30          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31          [string]$PythonExe = 'python',
32          [switch]$Pull
33      )
34
35      $ErrorActionPreference = 'Continue'
36      $date = Get-Date -Format 'yyyy-MM-dd'
37      $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38      $logDir = Join-Path $reportDir 'logs'
39      New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40      $log = Join-Path $logDir "$date.log"
41      $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
42
43      function Write-Status([string]$status, [int]$code, [string]$detail) {
44          $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45          "$stamp  status=$status exit=$code  $detail" | Out-File -FilePath $statusFile
-Encoding utf8
46      }
47
48      $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49      $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
50
51      # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
52      if (-not (Test-Path $checker)) {
53          "[$date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
| Tee-Object -FilePath $log
54          Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
55          exit 0
56      }
57      if (-not (Test-Path $manifest)) {
58          "[$date] golden manifest not present at $manifest ? skipping (corpus absent on this
machine)." | Tee-Object -FilePath $log
59          Write-Status 'SKIPPED' 0 'golden corpus absent'
60          exit 0
61      }
62
63      Set-Location $RepoRoot
64      if ($Pull) {
65          "[$date] git pull --ff-only" | Tee-Object -FilePath $log -Append
66          & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
67      }
68
69      $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
70      "[$date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
-Append
71
72      & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
73      $code = $LASTEXITCODE
74
75      switch ($code) {
76          0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
77          1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
78              "[$date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
Object -FilePath $log -Append }
79          4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
80          default { Write-Status 'ERROR' $code "checker error ? see $log" }
```

```
81      }
82
83      exit $code
84
```

44. assistant (2026-06-18T01:29:53.546Z)

This wrapper invokes `scripts/nightly_regression.py` which is excluded ? the wrapper itself looks correct (status mapping is consistent with the exit codes documented).

Now let me look back at `main.py` for the server startup. At line 862: `if args.server or not args.video_path:`. Earlier, line 791: `if args.video_path and not args.server:`. So if `--video-path` is provided AND `--server`, both branches: CLI processing block (line 791) is skipped (because `not args.server` is False), and then the server block runs. That seems intended.

But there's a subtle issue: when `args.server` is set WITH `args.video_path`, the server starts but the video is never processed. That's arguably fine ? server mode takes precedence.

Let me look at a potential real bug in `_drain_due_jobs` / global `db_instance` being None. Actually, let me reconsider the `run_phase3_eval.py` `--limit` interaction with `--only`. Line 113: `if args.limit:` ? if `--limit 0` is passed, `args.limit` is falsy so the limit is ignored, processing ALL. That's a minor off-by but `--limit 0` is an odd input. Let me set aside.

Let me focus on something more concrete. Let me re-examine the `download_highlight` endpoint ordering: it validates filename, then checks video exists, then resolves under root. There's a `media` determination. This looks fine.

Let me look at the `compile_highlights` `int(sc)` conversion ? `meta.get("shot_count")` could be a float string. `int("3.5")` raises ValueError, caught. Fine.

Let me now dig into `train_final` and `gate_verdict` more carefully for the paired sign test. In `gate_verdict` line 118:
```python
paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if r["video"] in per]
```
`per` is `floor.get("per_video", {})`. If floor's `per_video` is a list rather than dict... but the test uses a dict. Let me verify `eval_res["per_video"]` items have `"video"` and `"f1"` keys. Let me check `lovo_run` in `rally_seq_proto`.

45. user (2026-06-18T01:29:55.477Z)

```
53:def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
101:def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
109:def run(specs: List[Dict]) -> dict:
140:        results.append({"video": test, "thr": float(thr), "auc": auc,
141:                        "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
147:    mean_f1 = sum(r["f1"] for r in results) / len(results)
149:    return {"per_video": results, "mean_f1": mean_f1}
```

46. assistant (2026-06-18T01:30:05.803Z)

Line 147: `mean_f1 = sum(r["f1"] for r in results) / len(results)` ? if `specs` is empty, `len(results)` is 0 ? ZeroDivisionError. But that's the `run` function, used in harness. An empty manifest would crash there. Let me check whether `lovo_run` (which is `run`) guards empty. Line 109-149. If `specs` is empty, `results` is empty and line 147 divides by zero. This is reachable via `harness.run` with an empty manifest, but it's an edge case in a training tool and arguably the manifest is always non-empty. Not strong enough.

Let me reconsider the `gate_verdict` `floor_passed` logic more carefully. Line 114: `floor_passed = eval_res["mean_f1"] > floor_mean`. If `floor` is provided but has no `per_video` key, `per = {}`, `paired = []`, no sign test. Fine.

Let me now look at the `_seg_f1` threshold tie-break in `train_final`. `max(THRESHOLD_GRID, key=...)` returns the first max. Fine.

Let me step back and look at the actual FastAPI request validation more carefully. The `ProcessRequest.max_frames` is `Optional[int]`. There's no validation that it's positive. Negative max_frames could cause issues downstream, but that's the segmenter's contract.

Let me re-examine the `upload_part` total byte accounting bug more carefully ? this is the most promising area. Look at this scenario: part_bytes limit and the streaming. Actually, let me reconsider the case where overflow is detected but the LAST counted chunk was NOT written. `received` counts it, but `st["bytes"]` is only updated on success path (after overflow check, we abort and never reach the `st["bytes"] += received` line). So no accounting bug.

Let me look at `safe_output_filename` and `validate_input_path` for completeness, and at `resolve_under_root`.

47. user (2026-06-18T01:30:06.320Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Optional
4
5
6       def safe_output_filename(name: str) -> str:
7           """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9           `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12          """
13          if not name or not name.strip():
14              raise ValueError("output_filename must not be empty")
15          if "\x00" in name:
16              raise ValueError("output_filename must not contain a null byte")
17          if "/" in name or "\\" in name:
18              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21          return name
22
23
24      def resolve_under_root(path: str, root: str) -> str:
25          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26          rp = os.path.realpath(path)
27          rr = os.path.realpath(root)
28          try:
29              contained = os.path.commonpath([rp, rr]) == rr
30          except ValueError:  # different drives on Windows
31              contained = False
32          if not contained:
33              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34          return rp
35
36
37      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41          so the local 'point it at any of my videos' workflow is preserved. Setting an
42          allowed_root (a future hosted/tenant deployment) enforces containment.
43          """
44          if "\x00" in path:
45              raise ValueError("video_path must not contain a null byte")
```

```
46          if allowed_root:
47              resolve_under_root(path, allowed_root)
48
```

48. assistant (2026-06-18T01:30:14.306Z)

These path helpers are robust. Now let me reconsider `download_highlight`. After `safe_output_filename`, the name has no path separators. Then `os.path.join(output_dir, name)` and `resolve_under_root`. Fine.

Let me look more carefully at one thing in `main.py` that caught my eye: the `_finalize_reingest` and global `db_instance`. The module-level functions reference `db_instance` (global). In CLI mode this is set. In test contexts it's monkeypatched. Fine.

Let me reconsider `run_phase3_eval.py` line 138 again:
```python
already = db.query_play_segments(video_id, {}) or db.query_candidate_segments(video_id)
```
`query_play_segments` returns a list. `query_candidate_segments` returns a list. Empty list is falsy. So `already` is truthy if either has segments. Fine.

Let me look at the gen0 harness `run` function ? `train_final` calls `TrackNetRunner.parse_trajectory_csv(s["trajectory"])`. Let me check whether `lovo_run` and `train_final` both load the same data and whether there could be a frame_width/fps type issue. Actually, the test passes. Let me look at the `_seg_f1` used by `train_final`:

49. user (2026-06-18T01:30:15.308Z)

```
95                  merged[-1] = (merged[-1][0], e)
96              else:
97                  merged.append((s, e))
98          return [(s, e) for s, e in merged if e - s >= min_dur]
99
100
101     def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
102         f1s = []
103         for v in vids:
104             segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
105             f1s.append(score(segs, v["gts"], 0.5).f1)
106         return sum(f1s) / max(len(f1s), 1)
107
108
109     def run(specs: List[Dict]) -> dict:
110         data = {}
111         for s in specs:
112             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
113             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
114             X, times = features(arr, float(s["fps"]))
115             gts = load_golden_gts(s["labels"])
116             data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
117         names = list(data)
118         print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
119         print(f"features: {FEAT_NAMES}\n")
120
121         results = []
122         for test in names:
123             train = [data[n] for n in names if n != test]
124             Xtr = np.vstack([v["X"] for v in train])
125             ytr = np.concatenate([v["y"] for v in train])
126             clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
127             clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
128             # tune the decision threshold on TRAIN segment-F1 (no test leakage)
129             thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
```

```
float(t)))
130             v = data[test]
131             probs = clf.predict_proba(v["X"])
132             auc = roc_auc(v["y"], probs)
133             segs = probs_to_segments(probs, v["times"], float(thr))
134             sc = score(segs, v["gts"], 0.5)
135             # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
136             # from the threshold-transfer gap. Optimistic ? for diagnosis only.
137             othr = max(np.arange(0.2, 0.81, 0.05),
138                        key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
139             osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
140             results.append({"video": test, "thr": float(thr), "auc": auc,
141                             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
142                             "n_pred": len(segs), "n_gt": len(v["gts"]),
143                             "oracle_thr": float(othr), "oracle_f1": osc.f1})
144             print(f"[held-out {test}]  per-frame AUC {auc:.3f}  |  LOVO-threshold F1
{sc.f1:.3f} "
145                   f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train)  |  "
146                   f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
147         mean_f1 = sum(r["f1"] for r in results) / len(results)
148         print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
149         return {"per_video": results, "mean_f1": mean_f1}
```

50. assistant (2026-06-18T01:30:40.088Z)

Important finding: `lovo_run` (= `run`) uses `load_golden_gts(s["labels"])` at line 115, while `train_final` in the harness uses `load_gt_intervals(s["labels"], strict=False)` at line 87 and the manifest lineage uses `load_gt_intervals(s["labels"], strict=False)` at line 180. These are two DIFFERENT GT loaders. This could be a contract mismatch, but it may be intentional (proto vs harness). It's not clearly a bug.

Also note: `lovo_run` with a SINGLE video in the manifest ? `train = []` ? `np.vstack([])` raises ValueError. But again the LOVO contract implies ?2 videos. The harness `train_final` would handle a single video (Xall = single). The `run` would crash on n=1. This is an edge case, not a strong finding.

Let me now reconsider the most concrete area. Let me re-examine `main.py` `serve_index` ? it returns `f.read()` (a string) with `response_class=HTMLResponse`. Fine.

Let me look at the `get_config` endpoint: `return global_config` which is a Pydantic model. FastAPI serializes it. Fine.

Let me reconsider `upload_init` line 425: `max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024`. If `max_upload_mb` is configured as 0 ? `0 or 4096` ? 4096. Edge case but `or` handles falsy 0 by falling to default. Probably intended.

Let me now think about a genuinely concrete bug. Let me re-read the `upload_part` per-part limit logic vs the test `test_oversize_part_aborts_upload`. The test sends `b"x" * (1024*1024 + 10)` with `max_part_mb=1`. `part_bytes = 1*1024*1024 = 1048576`. TestClient may send the whole body as one chunk. `received = 1048586 > 1048576` ? overflow, abort. Returns 413.

But here's a subtle issue worth checking: in `upload_part`, the streaming reads chunks. With a small body that fits exactly at the boundary `received == part_bytes`, the check `received > part_bytes` is False (correct, allows exactly part_bytes). Good.

Let me look at whether there's an issue with `index` type. `@app.put("/api/upload/{upload_id}/part/{index}")` with `index: int`. FastAPI validates and coerces. A negative index? The check is `if index != st["parts"]`. `st["parts"]` starts at 0. A negative index would 409. Fine.

Let me reconsider the doctor.py `run_pip_install` ? it uses `--user` with no venv check. Not a bug per se.

Let me look at one more concrete area: `register_nightly_task.ps1` builds an inner script with string interpolation. If `$RepoRoot` or `$wrapper` contains a single quote, the generated PowerShell would break (injection/syntax). But paths with single quotes are rare and this is user-controlled local input. Not a strong defect.

Let me re-examine the `main.py` CLI trim path (lines 732-762) for a path bug. When `--trim-output` is an absolute path, `out_dir = os.path.dirname(out_path)`. If the absolute path is e.g. `C:\out.mp4`, `out_dir = "C:\"`. `os.makedirs("C:\\", exist_ok=True)` is fine. But if `--trim-output` is absolute with no directory like just a drive root... edge case.

Let me look much more carefully at whether `args.trim_start`/`trim_end` of value `0.0` is handled. Line 732: `if args.trim_start is not None and args.trim_end is not None:`. Good, uses `is not None` so 0.0 works.

But line 758 passes `[(args.trim_start, args.trim_end)]` ? if `trim_start > trim_end` it would produce a bad clip, but that's user error.

Let me check the `--debug-clip` path with `args.debug_clip` of value 0.0: line 764 `if args.debug_clip is not None:`. Good.

Now let me look at `run_cli_diagnose_clip` line 658-659 for an off-by issue with fps. If `fps` is 0 (cv2 fails to read fps), `start_frame = max(0, int((timestamp-3.0)*0))= 0`, `end_frame = min(total_frames, int((timestamp+3.0)*0)) = 0`. The loop `range(0, 0)` does nothing ? "No visual frames extracted". Handled gracefully.

Let me now look at the training `__init__.py` files and gen0 `__init__.py` for any import issues, and double check `register_nightly_task` principal line. Let me also examine the `run_phase3_eval` `entries = manifest.get("eval_sets", [])` ? if the manifest uses a different key, entries is empty, 0 targets. Graceful.

Let me reconsider one specific concrete bug candidate in `run_phase3_eval.py`. Line 91:
```python
db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
```
If `args.db` is a bare filename like `"my.db"` (no directory), `os.path.dirname("my.db")` is `""`, then `"" or "."` ? makedirs("."). Fine.

Let me focus and look at the `gen0/__init__.py` and `training/__init__.py`.

51. user (2026-06-18T01:30:42.815Z)

"""Owned-model TRAINING tree (ADR-008). backend/ must never import this package ?
serving consumes versioned artifact bundles only (CI-enforced; see training/README.md)."""
"""Gen-0 TAL harness (M0.4): train -> honest LOVO eval -> ship-gate -> artifact bundle."""
===LOAD_GT===
backend/eval/gt_loader.py:44:def load_gt_intervals(csv_path: str, *, strict: bool = True) ->
List[Interval]:

52. assistant (2026-06-18T01:31:08.009Z)

`load_golden_gts` is imported in `rally_seq_proto` from elsewhere. Both loaders exist and are used. This GT-loader divergence between `lovo_run` and `train_final` is a known proto-vs-harness thing, not a clear bug.

Let me now focus on one thing I want to verify concretely: the `gate_verdict` `floor_passed` vs the docstring/test. The test asserts `g["floor_passed"] is True` when `eval_res["mean_f1"] > 0.10`. That works.

Let me take a different, more targeted pass. Let me look at the FastAPI `download_highlight` for the subtle case where `name.lower().endswith(".mov")` determines media. But `safe_output_filename` already validated. Fine.

Let me reconsider whether there's a bug in `compile_highlights` when `req.segment_ids` is empty list. `matched_segments = [s for s in all_segments if s["id"] in []]` ? empty ? 400 "No matching

play segments". Fine.

Let me look at the `query_play_segments` endpoint and whether `req.video_id` being arbitrary is fine ? yes, just returns empty.

Now let me examine one final concrete candidate carefully: in `main.py`'s `_execute_processing`, the `finally` block at line 228 calls `emit_indexer_report` with `video_id` which is defined at line 138. But if `compute_video_id(req.video_path)` raises (e.g. file deleted between submit and run), `video_id` is never assigned, and... actually the `try` starts at line 153, AFTER `video_id` is computed at 138. The `finally` is part of that try/finally, so `video_id` is guaranteed assigned. Good.

But wait ? `val_results` and `val_context` are assigned at line 144 (inside the function but before the try at 153). If `registry.run_all_with_context` raises, the exception propagates out of `_execute_processing` before the try block, so no finally runs for the report. That's acceptable. But the `finally` references `val_results`, `val_context` ? both assigned at line 144 before the `try`. Good.

Let me look very carefully at the `upload_complete` flow for a resource issue. Line 515: `os.replace(st["tmp_path"], final_path)`. If `st["tmp_path"]` doesn't exist (e.g., aborted concurrently), raises. But the lock-protected `_uploads.get` and the in-process design make this unlikely.

Let me take yet another careful look at potential index errors. Let me check `report_to_dict` vs `aggregate` once more for the candidates stage where `mean_iou` might be absent... no, `as_dict()` always includes it.

Let me check the actual `nightly_regression.py` exit codes referenced ? no, it's excluded.

Let me reconsider the most likely real bug: the `_seg_f1` / threshold grid in `train_final` uses `THRESHOLD_GRID` (np.arange rounded) while `lovo_run` uses `np.arange(0.2, 0.81, 0.05)` inline. Both equivalent. Fine.

Let me now examine the `run_process_and_stitch.py` carefully ? line 117-122:
```python
meta = s.get("metadata", {})
if isinstance(meta, str):
    try:
        meta = json.loads(meta)
    except Exception:
        meta = {}
shot_count = meta.get("shot_count", 0)
```
If `meta` is None (metadata key present but value None), `s.get("metadata", {})` returns None (not the default, since key exists with None value). Then `isinstance(None, str)` is False, and `None.get(...)` ? AttributeError crash! Let me verify whether `query_play_segments` can return a segment with `metadata` = None.

Looking at `query_play_segments` line 425: `r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")`. So metadata is always a parsed dict (or whatever JSON), never None ? `json.loads("{}")` = `{}`. But could the stored metadata JSON be the literal `null`? `json.loads("null")` = None. Then `meta.get` crashes. But that requires metadata column to literally contain "null". Let me check how metadata is written. This is getting speculative.

But more importantly ? in `run_process_and_stitch.py`, `segments` may come from the SEGMENTER directly (line 81: `segments = segmenter.process_video(...)`), not from the DB. In that case segment dicts come from the segmenter's return value, where `metadata` could be a dict or absent. Let me check what the cache path returns: line 54 `segments = db.query_play_segments(...)` (metadata is dict). Line 81 segmenter returns dicts. The `_FakeSeg` in tests returns `"metadata": {}`. Real segmenters ? let me check if any return metadata=None.

This is too speculative. Let me instead verify the one concrete thing: does the codebase elsewhere guard `s.get("metadata") or {}` (the `or` pattern) rather than `s.get("metadata", {})`? In `main.py` line 599: `meta = s.get("metadata") or {}`. That's the SAFE pattern (handles None). But `run_process_and_stitch.py` line 117 uses `s.get("metadata", {})` ? the UNSAFE

pattern. This is a real inconsistency. However, `run_process_and_stitch.py` is a personal one-off script (hardcoded video path on line 24). Let me verify whether metadata can actually be None from the segmenter.

53. user (2026-06-18T01:31:08.995Z)

```
backend/pipeline/segmenters/gemini.py:466:                    "metadata": metadata
backend/pipeline/segmenters/motion.py:221:                    "metadata": metadata
backend/pipeline/segmenters/trajectory_hybrid.py:332:                        "duration": dur,
"metadata": metadata,
backend/pipeline/segmenters/yolo_hybrid.py:361:                        "metadata": metadata
```

54. user (2026-06-18T01:31:12.941Z)

```
                "detector": model_name
            }

            segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)

            if segment_id:
                segments_data.append({
                    "id": segment_id,
                    "start_time": start,
                    "end_time": end,
                    "duration": dur,
                    "metadata": metadata
                })

        return segments_data

---MOTION---
                    details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
                )

                # Store segment representation to return to caller
                segments_data.append({
                    "id": segment_id,
                    "start_time": start,
                    "end_time": end,
                    "duration": dur,
                    "server": server,
                    "receiver": receiver,
                    "metadata": metadata
                })
```

55. assistant (2026-06-18T01:31:30.484Z)

In these segmenters `metadata` is always a dict (constructed locally). So in `run_process_and_stitch.py`, the segmenter path always has dict metadata. The cache path uses `query_play_segments` which always produces a dict via `json.loads(... or "{}")` ? UNLESS the stored JSON is literally `"null"`. This is too speculative to be a confident finding. I'll drop it.

Let me take a final focused look at something I may have under-examined: the `process_video` 202 endpoint and whether `db_instance` could be None causing AttributeError, but that's startup-dependent and tests cover it.

Let me reconsider the `aggregate` function's `micro_f1` when precision+recall is 0 ? `_safe_div` returns 0. Fine.

Let me look one more time at a concrete reachable bug in `run_phase3_eval.py`. The `--detector` flag default is None, passed to `evaluate(... detector=args.detector)`. Fine.

Let me check the `--stage` choices include "auto" but `default_stages_for` handles "auto" by falling through. Wait ? `default_stages_for(args.segmenter, args.stage)` with `args.stage="both"` returns both. But the script's argparse default for `--stage` is "auto" and choices are `("candidates", "final", "both", "auto")`. All handled. Good.

Let me now verify there's not a genuine issue with `_arm_wake_timer` creating unbounded timers, but that's design, not a bug.

I've done a thorough pass. Let me do one final check on the gen0 harness `gate_verdict` ? specifically the `floor.get("per_video", {})` where floor's per_video maps video->f1 (float). Line 118 `per[r["video"]]` gives the float. `sign_test([lv for...], [fv for...])`. Correct. But there's a subtle issue: `floor_passed = eval_res["mean_f1"] > floor_mean` uses strict `>`. If they're exactly equal, floor_passed is False ? arguably correct (must BEAT the floor). Fine.

Let me verify one specific potential off-by-one: the gen0 manifest `created_at` uses `time.strftime("%Y-%m-%dT%H:%M:%S%z")` ? `%z` may be empty on some platforms but that's cosmetic.

Let me do a final scan of whether there's an unclosed resource. In `run_cli_diagnose_clip`, `cap = cv2.VideoCapture(...)`; if it raises mid-loop, `cap.release()` (line 680) is NOT in a try/finally ? a crash before line 680 leaks the capture. But cv2 ops rarely raise. Low confidence.

Let me verify the `_sha256` and file opens in harness are properly closed (they use `with`). Yes.

Based on my thorough investigation of scripts/, training/, and the FastAPI endpoints, I did not find any high-confidence, reachable defect that a maintainer would clearly agree is a bug. The code in this area has been heavily hardened. The few edge cases I found (empty-manifest ZeroDivisionError in `lovo_run`, single-video `np.vstack([])` in LOVO, the `s.get("metadata", {})` vs `or {}` inconsistency in the one-off `run_process_and_stitch.py`) are either guarded by the LOVO/manifest contract or not concretely reachable with realistic inputs.

Let me do one last verification on the most plausible candidate ? the `metadata` None case ? by checking whether `add_play_segment` could ever store the JSON literal `null`.

56. user (2026-06-18T01:31:31.935Z)

```
291:    def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
292-        try:
293-            with self._connect() as conn:
294-                cursor = conn.cursor()
295-                cursor.execute(
296-                    "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297-                    (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
298-                )
299-                conn.commit()
300-                return cursor.lastrowid
301-        except Exception as e:
302-            logger.error(f"Failed to add play segment: {e}")
303-            return None
304-
305-    def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
306-        try:
307-            with self._connect() as conn:
308-                conn.cursor().execute(
309-                    "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
310-                    (segment_id, timestamp, event_type, player, json.dumps(details or {}))
311-                )
```

```
312-                conn.commit()
313-            return True
314-        except Exception as e:
315-            logger.error(f"Failed to add event: {e}")
316-            return False
```

57. assistant (2026-06-18T01:31:40.256Z)

`add_play_segment` always stores `json.dumps(metadata or {})` ? never `null`. And `query_play_segments` does `json.loads(r_dict["metadata"] or "{}")`. So metadata from the DB is always a dict. Combined with segmenters always constructing dict metadata, the `s.get("metadata", {})` in `run_process_and_stitch.py` can never receive None in practice. That confirms it's not a reachable bug.

My investigation across the assigned area (scripts/ excluding nightly_regression.py, training/, and the FastAPI endpoints in backend/main.py) is complete. I traced every endpoint's request-validation, the argparse defaults and exit codes in the CLI scripts, the path/file handling, the upload byte-accounting, the async job lifecycle, and the gen0 training harness against their tests and the DB/result contracts. The code is heavily hardened and I found no concrete, reachable defect that meets the precision bar. Returning an empty candidates list.

58. user (2026-06-18T01:31:41.026Z)

Structured output provided successfully

59. assistant (2026-06-18T01:31:55.103Z)

I completed a thorough adversarial review of my assigned area and found no high-confidence, reachable defects.

What I traced (all read-only, against tests and DB/result contracts):

- **`backend/main.py` FastAPI endpoints** ? `/api/process` (202 + fast-fail validation), `/api/jobs/{id}`, the chunked-upload trio (`init`/`part`/`complete`/`abort`), `/api/highlights/{video_id}/{filename}` download, `/api/query`, `/api/compile`. Verified path-safety (`safe_output_filename`, `resolve_under_root`, `validate_input_path`), the upload byte-accounting and overflow/abort logic, the async job lifecycle (claim ? run ? terminal/parked), and destroy-after-confirm reingest. All consistent with `tests/test_upload_download.py` and `tests/test_async_jobs.py`.
- **`scripts/run_phase3_eval.py` + `backend/eval/batch.py`** ? argparse defaults, manifest walking, stage selection, and the `aggregate`/`report_to_dict` score-key contract (verified against `Score.as_dict()` and `tests/test_eval_batch.py`).
- **`scripts/run_process_and_stitch.py`, `scripts/doctor.py`** ? config load, cache path, shot-count filtering, diagnostics/exit behavior.
- **`scripts/*.ps1`** ? nightly wrapper exit-code mapping and the scheduled-task registrar.
- **`training/gen0/harness.py`** ? `sign_test` p-value math, `gate_verdict`, `train_final` threshold selection, artifact-bundle emission (verified against `tests/test_training_gen0.py`).

Candidates I considered and rejected as not concretely reachable:
- `run_process_and_stitch.py` uses the unsafe `s.get("metadata", {})` (vs `main.py`'s `s.get("metadata") or {}`), but confirmed metadata can never be `None`: `add_play_segment` stores `json.dumps(metadata or {})` and `query_play_segments` parses `... or "{}"`, and all segmenters construct dict metadata locally.
- `lovo_run` ZeroDivision / single-video `np.vstack([])` ? guarded by the LOVO/non-empty-manifest contract.
- `run_phase3_eval.py` `len(segs)` when a segmenter returns `Failure` ? live segmenters return bare lists, so unreachable.

No findings to report. Relevant files reviewed:
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\backend\main.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\backend\eval\batch.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-

base\scripts\run_phase3_eval.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\scripts\run_process_and_stitch.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\scripts\doctor.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\training\gen0\harness.py`